

Next-Gen Charting Blueprint (TradingView Lightweight Charts v5.1.0 Parity) ¹

Latest Stable Reference: As of Dec 2025, TradingView's Lightweight Charts library is at **v5.1.0**, which introduced major performance features like *data conflation* for large datasets ² ³. Below, we present a comprehensive capability parity matrix, detailed UX specs, and a full architectural plan to match and exceed v5.1.0 functionality.

1. Parity Matrix: Lightweight Charts v5.1 vs. Our New System

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Chart Types	Standard Time-based Chart: Uses Unix time or date strings on horizontal axis ⁴ ⁵ . Created via <code>createChart(...)</code> . Default 1 pane, time scale at bottom, right price scale visible ⁶ .	Instantiate chart, feed known date data; verify X-axis shows dates (or times if <code>timeVisible</code> true). Check one pane and right-side Y-axis present.	<code>Chart.create</code> chart by default bottom time a by default (lef mirror LC: one container, <code>la</code>
Chart Types	Yield Curve Chart: Horizontal scale is months (linear spacing), ignoring gaps (no weekends) ⁷ ⁸ . Created with <code>createYieldCurveChart()</code> . Specialized <code>yieldCurve</code> options (e.g. <code>baseResolution</code> , etc.) ⁹ .	Create yield-curve chart, input yields by maturity (e.g. 1,2,... months). Verify X-axis evenly spaced months and no blank dates. Crosshair/grid ignore missing months ¹⁰ .	<code>Chart.create</code> Internally use implementati (no gaps). Wil <code>baseResolu</code> weekends/ho spacing for m doesn't pause
Chart Types	Options (Price-based) Chart: X-axis is numeric price (not time) ¹¹ ¹² . Created via <code>createOptionsChart()</code> . Use case: option chain visuals, distributions.	Feed synthetic data (x=price, y=value). Verify X-axis ticks are numeric (no date) and reflect price values ¹³ . Ensure chart doesn't expect chronological order.	<code>Chart.create</code> clarity) to mal horizontal sca <code>HorzScaleB</code> numeric dom library's appr price internal plain number formatter).

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Chart Types	Custom Horizontal Scale Chart: LC provides <code>createChartEx(customBehavior)</code> for user-defined X-axis logic ¹⁴ .	Inject a stub custom scale behavior (e.g. stub that spaces points by value^2). Verify chart calls provided behavior for scaling.	<code>Chart.create</code> pluggable sca interface for h (index ↔ coordin design ensure injected and t logic.
Multi-Pane Support	Multiple Panes in One Chart: Available since v5.0 ¹⁵ . <code>chart.addPane()</code> adds a new stacked pane with its own Y-scale(s) ¹⁶ . Panes can be resized by user drag (if enabled) or via API (<code>setHeight</code> , <code>setStretchFactor</code>) ¹⁷ ¹⁸ . Pane separator styling is configurable (color, hover color) ¹⁹ .	Programmatic: call <code>chart.addPane()</code> , add series to it, verify independent scaling. User: drag separator – ensure adjacent pane heights adjust. Check <code>pane.setHeight()</code> API reflects in DOM.	Panled Chart multiple <code>Pan</code> <code>chart.addP</code> new pane API honor LC's de each ²⁰ . We pane dividers Resizing enab option). Each <code><canvas></code> f optional sepa
Pane API	Pane Management: LC <code>IPaneApi</code> offers methods: <code>setHeight(px)</code> ²² , <code>getHeight()</code> ²³ , <code>paneIndex()</code> ²⁴ , <code>moveTo(index)</code> reorder ²⁵ , <code>setStretchFactor()</code> to proportionally resize ²⁶ ²⁷ , etc. Can preserve an empty pane (don't auto-remove if no series) ²⁸ . Each pane can attach <i>primitives</i> (custom drawings) ²⁹ .	API test: create 3 panes, set stretch factors to 0.2/0.3/0.5 as in docs ³⁰ – verify relative heights 20%/30%/50%. Move pane order and ensure series moved with it. Remove all series in a pane with <code>preserveEmpty</code> disabled – pane should auto-remove.	Pane API Imp these method fixes pixel hei <code>pane.setSt</code> weight relativ <code>pane.moveTo</code> pane list and <code>pane.price</code> API for that p <code>preserveEm</code> (default) mea empty; true to Attach/detach custom draw plugin visual <code>pane.attac</code>

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Series Types	Line Series: Connects data points with straight lines ³² ³³ . Data: <code>{ time: T, value: N }</code> or gaps via <code>WhitespaceData</code> (time with no value) ³⁴ ³⁵ . Default styling: blue line, 3px, solid line ³⁶ ³⁷ . Supports <i>point markers</i> (disabled by default) and <i>crosshair marker</i> (small circle at intersection, default on) ³⁸ ³⁹ .	Feed sparse time series, verify line segments connect only adjacent real points (no line through large time gaps). Default style test: color = <code>#2196f3</code> (blue) ⁴⁰ , width = 3px ⁴¹ , solid. Enable <code>pointMarkersVisible</code> and ensure small points at each data (no overlap). Crosshair hovering a point should display a circular marker on the line ⁴² ⁴³ .	Series: Line - <code>chart.addS</code> creates a line interface: <code>{</code> <code>number }</code>. W <code>null</code> /white points as disc color <code>#2196f3</code> We support stepped (LC d implement op fill circles) via <code>pointMarke</code> and size <code>poi</code> Crosshair mar (draw small h under crossha customizable (defaults fallb
	Area Series: Fills area under line to zero line (or to next series below if stacked – LC's area is not inherently stacked, just base at time axis) ⁴⁹ ⁵⁰ . Data: same as Line (<code>SingleValueData</code>). Style: gradient fill defined by <code>topColor</code> and <code>bottomColor</code> + a border line (<code>lineColor</code>). Defaults: semi-transparent fill (blue) with matching line ⁵¹ ⁵² . Shares line options with Line (width, type, crosshair marker, etc.).	Test fill: provide line with peaks, verify area fill extends down to X-axis. Default style: check fill <code>topColor</code> ~<code>#2962FF</code> at 100% opacity, <code>bottomColor</code> ~<code>#2962FF</code> at ~28% opacity (from example) ⁵³ . Changing <code>topColor</code> and <code>bottomColor</code> should reflect in gradient. Crosshair marker on area: by default LC shows a circle on the boundary line (similar to line series).	Series: Area - filled polygon bottom of cha options: <code>lin</code> plus <code>topCol</code> We use vertic bottom colors solid to blue 2 features (step crosshair mar Baseline for a default, unless LC scope, but

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Series Types	Bar Series (OHLC Bar Chart): Depicts OHLC as a vertical line with left/right ticks ⁵⁴ ⁵⁵. Data: <code>{ time: T, open, high, low, close }</code> (LC uses <code>BarData</code> for OHLC) ⁵⁶ ⁵⁷. Style: <code>upColor</code> and <code>downColor</code> for bars where $\text{close} \geq \text{open}$ or $\text{close} < \text{open}$ ⁵⁸ ⁵⁹. No fill (just lines).	Test known OHLC: ensure bar's full height = high-low, with small horizontal ticks at open (left) and close (right) ⁵⁴. Color test: if $\text{close} \geq \text{open}$, entire bar uses <code>upColor</code> (and vice versa for down) ⁵⁹. If <code>open == close</code>, bar is flat – should it default to up or down color? (LC likely uses <code>upColor</code> if equal – assumption to verify). Verify no body drawn (just lines).	Series: Bar (Candlestick): line from <code>low</code> to <code>high</code> with ticks: left tick <code>open</code> ⁵⁴. We adopt <code>upColor = close > open</code> and <code>downColor = close < open</code> (match LC convention tests). No separate implementation for flat bars (length and line color explicitly; we follow LC's act
Series Types	Candlestick Series: Standard candlesticks: box from open to close, wick lines from high to low ³² ³³. Data: same as <code>BarData</code>. Style options: <code>upColor</code> (fill) & <code>downColor</code>, optional border colors, and wick colors. LC default: green up/red down bodies, with wicks colored same as body by default ⁶⁰ ³³. <code>borderVisible</code> toggles outlining the candle (default border same as fill if visible).	Test a rising candle ($\text{open} < \text{close}$): drawn filled with <code>upColor</code>, no border (if <code>borderVisible=false</code> by default in example) ⁶⁰. Falling candle: filled with <code>downColor</code>. Wick lines extend to high/low. If <code>wickUpColor</code> / <code>wickDownColor</code> set, verify wick adopts those (LC uses same as body in example) ⁶⁰. Turn <code>borderVisible=true</code> and set <code>borderUpColor</code> / <code>borderDownColor</code>: confirm border stroke around body.	Series: Candlestick: from open to close, wick lines from high to low. candle, else "close". Default color: <code>#26a69a</code> (teal). Wicks: lines from open to high/low. wick color = body color. <code>wickUpColor</code> / <code>wickDownColor</code> specified ⁶⁰. default <code>borderVisible=false</code> API (LC might follow LC's act same as fill or body color. example immutability. follow LC's act <code>true</code> with border. border is not visible by default (custom). User can set <code>borderDownColor</code> automatic (background color). support a fixed border color needed (for consistency terms).

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Series Types	Histogram Series: Series of columns where each data point is a vertical bar from zero to value ⁶¹ ⁶². Data: <code>{ time: T, value: N, color?: string }</code> (each point can override color) ⁶³. Style: a single default <code>color</code> for all bars (unless overridden per point). No outline by default.	Provide sequential data. Verify each point draws a filled rectangle from baseline to value. If some data points include a <code>color</code> property, confirm those bars render in that color ⁶³ while others use series default color ⁶⁴. Check baseline – LC histograms start at 0 (for positive/negative values above/below? – LC histogram doesn't explicitly mention negative handling; assume it draws upward from 0).	Series: Histogram bar graph. De (green) ⁶⁴. E specify a <code>col</code> item color if p negative valu (we'll treat 0 a is negative, w – but likely sin No border un Column width available inter
Series Types	Baseline Series: An area chart split by a “baseline” value (two-tone fill) ⁶⁵ ⁶⁶. Data: same as single-value. Requires <code>baseValue</code> (default 0) which can be a price or percent reference ⁶⁷. Style: separate fill colors for above baseline vs below baseline, and separate line colors for the baseline line itself. Example shows green fill above and red fill below a baseline of 25 ⁶⁶ ⁶⁸.	Create baseline series with <code>baseValue</code> X. Provide points above and below X. Verify region above is filled with “top” colors and below with “bottom” colors ⁶⁶. Ensure a horizontal baseline is drawn at value X. Test <code>baseValue</code> type “percent” (if supported) by feeding percent mode – maybe baseline moves with first value (LC supports baseline type 'price' vs 'percent' ⁶⁷).	Series: Baseline series that sp Options: <code>bas</code> 'price', pr 'percent', supports 'pric percent, base value. Style: u <code>topFillColor</code> <code>bottomFillColor</code> above and be <code>bottomLine</code> segments abo ⁶⁸. Default k percent (mean We will draw t above baselin its own color. line at the bas colored with t can simply dr data intersect example sugg below differen <code>topLineColor</code> above baselin when below (crossing).

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Improvement
Custom Series (Plugins)	Custom Series API: LC allows defining new series types via the plugin interface. A custom series provides an <code>ICustomSeriesPaneView</code> (drawing logic) and is added with <code>chart.addCustomSeries(paneView, options)</code> ⁶⁹ ⁷⁰. Custom series can define their own data format and rendering (e.g., indicators).	Use an example plugin (e.g. a custom histogram from LC docs) and add to chart ⁷¹. Verify it renders and interacts (hover, etc.) as expected. Custom series should behave like native series in updates and scaling.	Custom Series API: an interface <code>ISeriesRenderer</code> for developers implementing <code>drawOnCanvas</code> and <code>chart.addSeries(options)</code> with implementing the <code>ISeriesRenderer</code> interface. We will call its <code>render</code> method. All core services will provide coordinates, and for custom series we will provide custom series scale behavior. We will provide a simple custom series (e.g. shapes) to ensure it is sufficient.

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Series Defaults & Common	Series Common Options: All series share common settings: <code>visible</code> (true by default) ⁷² ⁷³, <code>lastValueVisible</code> (price label on scale, true by default except <code>yieldCurve</code>) ⁷⁴, <code>title</code> (empty default) ⁷⁵, <code>priceScaleId</code> (which Y-scale to attach; default 'right' if it exists, else 'left' ⁷⁶), and crosshair marker toggles (for Line/Area) ⁴³. Also <code>priceLineVisible</code> (draw horizontal line at last price, true by default except <code>yieldCurve</code>) ⁷⁷, with style and color options, and optional last-price animation (flash or move) which default is disabled ³⁶.	Turn off <code>series.visible</code> and ensure it fully hides (no line/markers, no last price on scale) ⁷². Toggle <code>lastValueVisible</code>: when false, last price scale label disappears ⁷⁴. Assign <code>title</code>: ensure it appears next to last value label in axis (LC shows series title in the axis label background) ⁷⁸. If multiple series share one price scale, each series title appears on its last value label (in LC they concatenate e.g. "(Series A)" on label) ⁷⁸. Test last price line: should be a horizontal line across chart at last price when enabled, style dashed by default ⁷⁹ ⁸⁰.	Series Comm <code>series.set</code> drawing and Default visible label on the p default (except noted ⁸¹). W set) inside tha styled similar assignment: b primary right left ⁸². We in option so the scale (e.g. '1' custom string group). Our s create a new v custom id is u which are alw Price Line: we horizontal line default true) v custom, width multiple serie multiple last l (distinguished <i>animation</i>: we fade or blink <code>LastPriceA</code> Disabled (0) b remains no an the last-value change.

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Price Scales & Axes	Multiple Price Scales: By default each pane has one <i>right</i> price scale (visible) and an optional <i>left</i> (hidden by default) ⁸⁴. A series can target <code>'left'</code>, <code>'right'</code>, or any custom scale id ⁸⁵. Custom scales (overlays) are invisible axes that auto-scale to that series data ⁸³. LC price scales have independent options but share the same space if both visible (they auto-adjust width if both sides on).	Test adding two series to same pane, one on 'right', one on 'left': enable left scale visible, ensure each series' last price appears on its respective side ⁸⁶. Add a third series with a unique <code>priceScaleId</code>: verify it doesn't show an axis (overlay) but its values are auto-scaled (e.g., extremely different value range still renders in view) ⁸³.	Scales Archi a right and le allow <code>chart</code> <code>'right')</code> to Additional sca <code>series.pri</code> we create an Overlay scales enforces auto We handle ax are visible, we both. Each sca to fit labels (w width each up ⁸⁸. Scales sh height). Series share one sca all their data.

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Price Scale Options	PriceScale Modes: Linear (Normal) vs Logarithmic vs Percentage vs IndexedTo100 ⁸⁹ ⁹⁰ . Default mode = Normal (linear) ⁹¹ . In Percentage mode, first visible price is 0% ⁹⁰ ; in IndexedTo100, first visible becomes 100 ⁹² . Auto-scale: true by default (scale adjusts to visible data range) ⁹³ . Invert: false by default (if true, flips Y-axis upside down) ⁹⁴ . Align Labels: true (prevents label overlap by adjusting drawing) ⁹⁵ . Scale Margins: default top 0.2 (20%), bottom 0.1 ⁹⁶ – extra space above/below highest/lowest point.	Set <code>priceScale.mode = Logarithmic</code> on a series and verify equal ratio distances on axis (e.g., values 10 and 100 are equidistant from 1 and 10). Test Percentage: ensure axis labels show “%” and relative changes (if first visible price is P, it shows 0% at P and other values as ±%). IndexedTo100: first visible bar’s value shows “100” and others as percentage of that (e.g. 120 means 20% up). Invert: set <code>invertScale</code> and confirm chart flips vertically (higher values at bottom). Margin: confirm by default there’s ~10% space below min and 20% above max in view ⁹⁶ . Turn off <code>autoScale</code> and set manual visible range via <code>priceScale</code> API – ensure no autoscaling occurs on data change.	Scale Modes: Mode is per price scale default = linear convert data values for pixel mapping non-positive values auto-switch to minimal positive Indexed100: values value (first visible) percentage mode values = ((price - min) / (max - min)) * 100 label with “%”. add 100 (so first visible is 100) accordingly) ⁹⁰ show % or not AutoScale: by default manual settings hold it until re-render implement by mapping (y = min + (max - min) * (value - min) / (max - min)) layout – if two price labels can hide one default true to will adopt a size two price labels overlaps; skip Margins: controls defaults 0.2 top margins (in pixels) the auto-scale them when canvas (expand min/

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Improvement
Price Scale Visual	Axis Visibility & Style: Right axis visible by default, left hidden ⁶. <code>visible</code> option can hide a scale entirely (e.g., hide right if you only want left) ⁸⁴. Axis line (border between chart and scale) is drawn by default (<code>borderVisible=true</code>) in color <code>#2B2B43</code> (a dark gray) ⁹⁹ ¹⁰⁰. Tick marks on axis: off by default (<code>ticksVisible=false</code>) ¹⁰¹. Text color defaults to chart layout <code>textColor</code> (unless overridden) ¹⁰². <code>entireTextOnly=false</code> by default (meaning it will display partial labels even if cut off at edges) ¹⁰³. There's also <code>minimumWidth</code> (0 by default) to reserve space ¹⁰⁴, and an option to ensure top/bottom tick marks visible (off by default) ¹⁰⁵. YieldCurve charts default to left scale visible instead ¹⁰⁶.	Turn off <code>rightPriceScale.visible</code> and turn on <code>leftPriceScale.visible</code>: verify axis appears on left with all labels, and right axis is gone ⁶ ⁸⁶. Check border: by default a separator line is drawn at chart right edge ⁹⁹. If <code>borderVisible=false</code>, confirm no line. Tick marks: enable <code>ticksVisible=true</code> and ensure little tick lines appear next to labels on axis ¹⁰⁷. <code>entireTextOnly=true</code>: if set, ensure axis labels that don't fully fit (e.g. big font at top) are hidden rather than partially drawn ¹⁰⁸. Test <code>minimumWidth</code>: set a large min (say 100px), ensure axis area doesn't shrink below that even if labels are small (use long numbers to test growth).	Axis Rendering: showing each (time-chart) or yield-curve chart visible, right chart implement a chart from axis ⁹⁹, using <code>borderColor</code> provided enabled, draw each grid line LC: off by default <code>ticksVisible=false</code> <code>priceScale</code> otherwise fall <code>textColor</code> ¹⁰² and give axis Only: if true, top/bottom unchopped text, partial (the default we respect a labels are narrow space up to the Marks: if enabled top and bottom point exactly implement by scale min and

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
Time Scale & X-Axis	Time Scale (X-axis): Shows dates by default (if chart spans ≥ 1 day) or time if intraday. Options <code>timeVisible</code> (false default) and <code>secondsVisible</code> (false) control detail ¹¹³ ¹¹⁴. If <code>timeVisible=true</code>, the axis and crosshair use <code>hh:mm</code> or <code>hh:mm:ss</code> format for intraday data ¹¹⁴ ¹¹⁵. The time axis can be in a specified timezone (LC supports <code>timezone</code> option, default UTC) ¹¹⁵. Default bar spacing $\sim 6\text{px}$ ¹¹⁶ and <code>rightOffset</code> = 0 bars ¹¹⁶. The chart naturally allocates some left margin for first point if few points present ¹¹⁷. Methods: <code>.fitContent()</code> auto-scales time to show all data ¹¹⁸; <code>.setVisibleRange(range)</code> or <code>.setVisibleLogicalRange()</code> to programmatically set view ¹¹⁹ ¹²⁰. Scrolling: by default, new data shifts chart (unless user scrolls back, then <code>rightBarStaysOnScroll</code> kicks in to freeze auto-scroll). LC's time scale also can <i>fix left/right edge</i> (prevent scrolling beyond data) via options (<code>fixLeftEdge</code>, etc.).	For an intraday series (e.g. minute candles), initially <code>timeVisible=false</code> means X-axis shows only date (no intraday time). Set <code>timeVisible=true</code>: verify axis and crosshair tooltip now show time of day ¹¹³. If data includes seconds granularity, set <code>secondsVisible=true</code> and ensure seconds show ¹¹⁴. Test <code>timeScale().fitContent()</code>: after zooming in, calling <code>fitContent</code> should zoom out to include all bars ¹¹⁸. Programmatically set a visible logical range beyond data start (e.g. start at -10 index): if <code>fixLeftEdge=false</code>, chart will allow empty space to left (then auto-adjust if scrolled) ¹²¹. With <code>fixLeftEdge=true</code>, it should clamp at index 0. Similarly test right edge.	Time Axis Im we display da daily data) an time-of-day fo is true. We pa timestamps o accepts both support a ti 'UTC' or ' using Intl.Dat Default to UT timezone or c Offset: We de px per bar ¹¹⁶ last bar is at c can adjust via <code>timeScale.</code> implement zo (increase = zo out). fitConte range coverin first and last p ¹¹⁸. Visible R <code>timeScale.</code> to) (by time variant. We h beyond data e <code>fixRightEd</code> options). Edge <code>fixLeftEdg</code> past first data <code>fixRightEd</code> point to right side empty sp <code>rightBarSt</code> default): if use data keeps ch scrolls left, ch bars (LC's defa

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Improvement
Crosshair Interaction	Crosshair Modes & Lines: LC crosshair has modes: <i>Normal</i> (free), <i>Magnet</i> (snap to data, default) ¹²⁴, <i>Hidden</i> (no crosshair drawn) ¹²⁵, and <i>MagnetOHLC</i> (snap to any OHLC point) ¹²⁶. Default = Magnet (snaps horizontal line to closest series value, usually last price of nearest bar) ¹²⁷ ¹²⁸. Crosshair displays vertical line at pointer's time and horizontal line at pointer's price. By default both lines are visible (style: color <code>#758696</code>, width 1, large-dashed) ¹²⁹ ¹³⁰. Crosshair axis labels: LC shows a label on price axis for the horizontal line value and on time axis for the vertical line, with inverted colors matching series (for price) and default style for time. Option <code>doNotSnapToHiddenSeriesIndices</code> (false by default) if true will ignore hidden series data when snapping ¹³¹.	Verify default snapping: on an OHLC series, move crosshair near a bar – horizontal line should lock to that bar's close (Magnet mode) ¹³². Switch to Normal mode: crosshair moves exactly under cursor (no snapping). Hidden series snapping: have two series, hide one; in magnet mode, ensure crosshair snaps only to visible one when <code>doNotSnapToHiddenSeriesIndices=true</code> ¹³¹. Style: confirm crosshair lines drawn in default color ¹²⁹, 1px dashed. Toggle <code>vertLine.visible</code> or <code>horzLine.visible</code> false – those lines disappear but crosshair marker on series still shows (LC note: disabling lines does not disable the small markers on lines/areas ¹³³). Crosshair labels: ensure price label shows series value (with series color band or text) and time label shows date/time.	Crosshair Improvement to Magnet mode to find nearest OHLC point. For OHLC series, the Close of the bar. Magnet definition: snap to OHLC value (close) (if enabled via <code>doNotSnapToHiddenSeriesIndices</code>). OHLC value (close) include Hidden series. Crosshair lines: draw vertical line in all panes) and horizontal line in pane under cursor. Style: color <code>#758696</code>, width 1, LargeDashed. Sub-options: <code>crosshair.color</code>, <code>crosshair.width</code>, <code>crosshair.dashed</code>, <code>crosshair.vertLine.visible</code>, <code>crosshair.horzLine.visible</code>. If line/area is hidden, the crosshair line/area by series color is implemented. We will draw a small marker on series scale at the crosshair position. Visible price series color will match series color convention (LC price) and text on time axis, we will format time as crosshair marker by redrawing. Snap to Hidden Series magnet will continue to snap to hidden series <code>doNotSnapToHiddenSeriesIndices</code> (analogous to <code>doNotSnapToHiddenSeriesIndices</code> if enabled, hidden series snapping ¹³¹).

Pointer & Zoom UX

Scrolling & Zooming Controls: LC allows:

drag chart to scroll horizontally (left mouse drag by default) ¹³⁵, scroll with mouse wheel (enabled by default) ¹³⁶, and pinch-zoom on touch (enabled) ¹³⁷. These are configurable via `handleScroll` and `handleScale` options. By default: *MouseWheel scroll* = true ¹³⁶, *Drag scroll* = true (`pressedMouseMove: true`) ¹³⁵, *Touch drag* = true for both horiz and vert gestures (LC intercepts vertical swipe to avoid page scroll if `vertTouchDrag=true`) ¹³⁸ ¹³⁹. Zooming: *Wheel zoom* – LC actually treats mouse wheel as scroll, not zoom, by default (but `handleScale.mouseWheel` exists to allow zoom instead ¹⁴⁰). *Pinch* (two-finger) = true (zooms X and Y as appropriate) ¹³⁷. *Drag scaling* – dragging axis: enabled (you can drag price axis to scale vertically, time axis to scale horizontally) ¹⁴¹ ¹⁴². Double-click axis resets that scale (e.g. double-click price axis = auto scale price range) – LC has `AxisDoubleClickOptions` to toggle this behavior (default on). *Kinetic/Inertia*: on touch scroll, LC has inertia enabled (touch: true) ¹⁴³; for mouse drag, inertia is off by default (mouse: false) ¹⁴⁴.

Drag test: click-drag chart background – chart should scroll in time. Mouse wheel: scroll a notch, chart should pan left/right (one notch ~ one bar by default). If

`handleScale.mouseWheel=true` (not default), then wheel should zoom instead (increase/decrease `barSpacing`). Pinch on trackpad or touchscreen: verify smooth zoom in/out (if two-finger pinch). Drag price axis: candles stretch/squash vertically (changing price scale). Drag time axis: zoom in/out (`bar spacing changes`) – ensure this matches `axisPressedMouseMove.time` setting ¹⁴¹. Double-click price axis: price scale should autoscale to fit data ¹⁴⁵; double-click time axis: time scale resets (fit all or cancel zoom/scroll offsets). Inertia: flick scroll on touch device, release – chart should continue moving briefly (LC kinetic). Ensure this stops gracefully and doesn't overscroll past data (or bounces if allowed?).

Pan & Zoom

full set of inter defaults: **Pan** horizontally (Mouse wheel scroll timeline wheel by default option to treat (`handleScale`). One-finger dr data, vertical allow vertical `vertTouchD` both axes ¹³⁹. Two-finger pin two-finger *pa* should act as center). **Axis** dragging axes (`axisPresse` `price=true` capture mous vertical drag `priceScale` (sc horizontal dra `barSpacing` (z We will imple axis to reset t manual zoom ¹⁴⁵. Double- all data (or las These are cor via options). I fling scrolls, w decelerating (touch ¹⁴³). W scrolling: capt and continue mouse, LC lea off as well, bu `kineticScr` We'll expose (default true) `kineticScr` to match LC 1 prevent overs scroll past dat

Capability	Lightweight Charts v5.1 Behavior & Defaults (sources)	Test Method (Verification)	Planned Imp
			ends (or impl desired – but

Sources: The above behaviors and defaults are drawn from official LC documentation and release notes, ensuring factual parity. Key references include LC’s **Series documentation** ⁴⁹ ³² , **API reference** for options and defaults ⁹⁹ ¹²⁹ , and **Release Notes v5.0+** for multi-pane and plugin details ¹⁵ ¹⁴⁸ .

2. UX & Interaction Specification

Chart & Axis Behavior: The chart will feel **smooth and interactive**, maintaining 60fps even during rapid updates or drags. Panning is anchored to data: dragging the plot area moves the time scale linearly. If the user scrolls beyond the oldest data, the chart will display a blank space on the left until `fixLeftEdge` option stops it at the first data (clamped if enabled) ¹⁴⁹ . Similarly on the right, by default the chart can be scrolled forward in time (empty future space) unless `fixRightEdge` is true ¹⁴⁹ . When new real-time data arrives and the chart is at the rightmost position (tracking latest), it will auto-scroll to keep the last bar in view. If the user has scrolled away (chart not at right edge), new data will not jerk the chart – it will *stay in place* (user must manually scroll or call `timeScale().scrollToRealTime()` to catch up, emulating TradingView’s “auto-scroll off” when you look back).

Zoom/Pan Interactions: Mouse wheel scrolling on the chart will, by default, pan the chart horizontally by a few bars per notch (providing fine-grained control). If the user holds `<kbd>Ctrl</kbd>` (or if we enable an option), the wheel can instead zoom the time axis (this mimics many charting UIs). Dragging the **time axis** itself will dynamically zoom in/out (horizontally) at the point of drag – our system will implement this exactly: clicking and moving left/right on the time axis recalculates `barSpacing` in real-time, anchored at the drag start point ¹⁴¹ . Dragging the **price axis** scales the price—moving up compresses (zoom out vertically), moving down expands the scale (zoom in) ¹⁴² . This gives immediate feedback as candles stretch or squeeze. Double-click on price axis will reset to full autoscale (restore default nice margins) ¹⁴⁵ ; double-click on time axis will fit all time points in view (or reset zoom to default range) for quick re-centering.

Crosshair & Tooltip: The crosshair is always drawn on the top layer, updated with no lag. In normal mode it moves freely under the cursor, showing interpolated values if between data points (for line/area it will interpolate value; for OHLC it will just show nearest). In magnet mode (default), the crosshair “jumps” to the nearest data index’s value on mousemove – for example, hovering near a candlestick snaps the horizontal line to that candle’s close (or nearest OHLC if MagnetOHLC mode) ¹⁵⁰ . The vertical crosshair line will snap exactly to the candle’s time. The chart will display **crosshair tooltips**: one on the price scale indicating the value at crosshair, and one on the time axis indicating the time. Our system will improve their design: we’ll use smooth fading or slight movement animation to make the tooltip feel “attached” to the crosshair with minimal jitter. The price tooltip will adopt the series’ color or a distinct style if multiple series share it – e.g., if two series on the same price scale are both visible, we might show multiple values in one tooltip or separate stacked labels (in LC, if series have titles, the last price label shows e.g. “SeriesA 123.45” ⁷⁸ ; we can extend that to crosshair as well). The time tooltip will format according to interval: date for daily points, time for intraday, including seconds if enabled ¹¹⁴ ¹¹³ . If the chart’s timezone is set, the tooltip reflects it (e.g., “2025-12-24 14:30:00 EST”).

Hover Markers: For line and area series, we will draw a small hover circle at the exact data point under the crosshair (if the crosshair is snapped to data). This marker uses the `crosshairMarker` style (by default a filled white border around a point) ¹⁵¹ ⁴⁸. This provides a clear visual for the user's focus. The crosshair lines themselves are subtle (dashed gray by default) so as not to overpower the chart, but easily visible against both light and dark backgrounds (the color #758696 is chosen for sufficient contrast on typical backgrounds ¹²⁹). Users can customize crosshair line color, style (solid, dashed, dotted) and width; for example a thicker crosshair might be useful in certain UIs.

Touch Gestures: On touch devices, one-finger drag will scroll the chart horizontally. If `vertTouchDrag` is enabled (default true) ¹³⁹, dragging vertically with one finger will also move the chart up/down *only if* the user is at an extreme zoom where vertical scrolling is meaningful (in most financial charts vertical drag within the pane has no effect aside from maybe panning an indicator that doesn't auto-scale – we may simply disable vertical single-finger drag to allow page scrolling in a mobile browser). Two-finger pinch will zoom the time axis. We will implement pinch-zoom focusing on the midpoint between fingers as the zoom center (so it's intuitive). Also, two-finger drag (two fingers moving together) will pan the chart without zooming (we can detect pinch vs pan by the distance change between fingers – if distance stays roughly same, it's a pan gesture). These behaviors make the chart feel like a native mobile chart (LC similarly handles horizontal and vertical gestures separately ¹³⁸ ¹³⁹).

Kinetic scrolling: Flicking the chart (fast drag and release) on touch will trigger kinetic scroll – the chart continues moving and decelerates smoothly. We will calibrate this so that a moderate flick scrolls a bit further beyond release point, but friction ensures it stops before or exactly at data bounds (no endless drift). If the user flicks to an empty region beyond data (with `fixLeftEdge=false` for example), we can allow a slight overscroll with a bounce back. However, by default we'll clamp to prevent seeing past the first/last data point – instead, kinetic energy will dissipate as it reaches the edge, stopping right at the end (preventing blank space if `fixEdge` is on, or minimal blank if off). On desktop, because kinetic is off for mouse by default ¹⁴⁷, dragging rapidly just stops when the mouse is released; we might allow enabling inertia for mouse as an option for a more “trackpad-like” feel.

Resizing & High DPI: When the chart container is resized, our chart will auto-resize (if `autoSize` is true) ¹⁵², maintaining content scaling. We will use `ResizeObserver` (like LC does) to detect container changes ¹⁵² and redraw accordingly. On high DPI screens, our canvas will use `devicePixelRatio` scaling to keep lines and text crisp (i.e., we size canvas larger and scale down via CSS). We'll test that text and thin lines (like crosshair or grid) remain sharp and not blurry on 2x DPI – by adjusting canvas pixel ratio and re-measuring text. This matches LC's practice of rendering in device pixel ratio for sharpness.

Grid and Labels: Grid lines (if enabled) will scroll/zoom with the data (they are tied to time and price ticks). During zoom or resize, grid lines will adjust dynamically but without stutter – we'll calculate appropriate tick intervals (maybe using LC's weight system for time ticks to decide when to show month labels vs days, etc.). The price axis will highlight the last price of each series with a colored label that moves as the price moves in real-time. When new data arrives for the latest bar, if last price animation is enabled, we might briefly flash the label border or background (green/red for up/down) – providing a subtle cue of update (LC's `lastPriceAnimationMode` can blink or last tick mark movement; we can implement a fade).

Edge Cases & Golden Behaviors: - If there are **huge numbers of points** (millions) and the user zooms all the way out, our conflation/downsampling will kick in to ensure performance (see Perf Plan below). From a UX perspective, the user will still be able to zoom smoothly – instead of freezing, the chart will show

aggregated data. This will be mostly unnoticeable except extremely zoomed out: the chart line might look smoothed (this is acceptable as LC 5.1 conflation does similarly when points < 0.5px apart ¹⁵³). - If new data arrives at a high frequency (e.g. 10k points/sec streaming), our chart will batch the visual updates to, say, 60Hz. The user sees a smoothly updating line rather than thousands of discrete step updates (the internal data is buffered). This preserves smooth motion and keeps latency low (only ~16ms). - **Tooltip coherence:** If multiple series share a time, crosshair tooltip will list values for each series. We ensure that if series have drastically different scales (e.g. one in thousands, another in single digits), the tooltips don't overlap – possibly we position multiple price labels side by side or stack them with minimal overlap. This is an improvement in our system: whereas LC shows only one combined label for one scale, we'll handle multiple series on same scale by combining, and multiple scales by multiple labels (so you might see one label on left for one series, one on right for another). - **Annotations/Markers:** If the user adds markers (via our plugin or API) like icons or notes on specific bars, these will hover above the chart. Our interaction model will treat them as part of top layer: they won't interfere with crosshair (crosshair passes under them). If a marker is clicked, we'll fire a marker-click event (for custom callbacks). - **Legend updates:** If an external legend is used (some users show the latest values of series elsewhere), our API's new `subscribeDataChanged` event can be used – it will emit an event whenever a series' data changes (append or update) ¹⁵⁴. This helps in building rich legends without polling.

In summary, all interactions will be **snappy** and intuitive, mirroring or exceeding TradingView's lightweight charts. Default behaviors are carefully chosen to match user expectations (e.g., scroll wheel pans by default to avoid accidental zoom chaos, drag to scroll, double-click resets, pinch zooms, etc.), with options to customize to specific needs. All these will be covered by user-focused documentation and a set of automated interaction tests (e.g., synthetic wheel and drag events to ensure proper zoom/pan) to prevent regressions.

3. Architecture Deconstruction

High-Level Architecture: Our chart is organized in layered modules responsible for **Data, Transforms (scales & coordinates), Rendering, Interaction/Events**, and **Plugin Extensions**. We draw inspiration from Lightweight Charts' internal separation of model vs view ¹⁴⁸ and add clear boundaries for performance.

- **Data Model Layer:** At the core, we have a `DataStore` for each series. This stores time-value points in sorted order (we will use binary search for fast indexing and retrieval). For memory efficiency, we can use typed arrays or packed objects for large series. The data store exposes methods to add/append data and to retrieve data windows by index or time. It also can produce **conflated (aggregated) data** for large spans – implemented via a strategy pattern (e.g. `conflate(fromIndex, toIndex)` can return a reduced set) ¹⁵⁵. All series data updates go through a `SeriesDataManager` which notifies other components (price scale for autoscaling, etc.). If historical data is chunk-loaded, a `DataProvider` module will handle fetching additional points when the user scrolls past loaded range, and evicting old points if needed (to manage memory for extremely large datasets). This modular `DataProvider` means the chart can show endless data by loading on demand – analogous to how some systems page historical data.
- **Coordinate Transforms:** Similar to LC, we maintain **TimeScale** and **PriceScale** objects. The **TimeScale** converts time (or index) to logical coordinates (floating indices) and then to pixel X coordinates ¹⁵⁶. It holds the current `LogicalRange` (float from-to) of visible bars ¹⁵⁶. It also handles tick mark generation for the time axis (deciding which timestamps to label – e.g. major vs

minor ticks using weight logic akin to LC's tick mark weights). The **PriceScale** for each axis converts values to Y coordinates. Each PriceScale tracks its data range (min/max of visible series data, updated each draw or on data change) and has a method to apply mode (linear/log) and margins ¹⁵⁷. Data flow: The DataStore informs PriceScale when new data enters or visible range shifts (so PriceScale can recalc min/max if autoscale). PriceScale outputs a transform (like a function `value => y`) used by the renderer. Importantly, price scales have references to the series that use them, so they can aggregate all their data for autoscale.

- **Layout & Pane Manager:** A `ChartPane` class manages one pane's content. It contains one or more **SeriesPaneView** objects (each series' prepared drawing primitives) and any **PanePrimitives** (custom drawings or plugins, like watermarks, that belong to that pane). The Pane is responsible for layering: series are drawn in a specific Z-order (in LC all series appear under the crosshair, and possibly under annotations; we'll maintain a similar layering: e.g., grid -> series -> series markers -> crosshair -> overlays). The Pane also holds its dedicated PriceScale(s) and communicates with the Chart's TimeScale for horizontal coordinates. If multiple panes exist, a `ChartLayout` orchestrates them (stacking vertically, handling relative stretch factors for resizing ¹⁵⁸).
- **Rendering Pipeline:** The rendering is largely done on HTML5 Canvas 2D (one canvas per pane for series, plus possibly separate canvas for top-layer UI like crosshair and axes, or we can also choose to draw everything in one canvas per pane – we might use layers for clarity: e.g., one canvas for series+grid, one on top for crosshair+axes to simplify z-index and update isolation). At each animation frame or event, a **RenderCoordinator** checks if any part of the chart is invalidated and needs redraw. The basic flow:
 - Input events or data changes set flags (e.g., series data changed -> invalidate series and maybe axes; zoom/pan -> invalidate axes and series positions; resize -> invalidate everything).
 - On next frame, the coordinator computes new scales (TimeScale and PriceScales recalc internal state).
 - Then for each Pane, it asks each SeriesPaneView to draw. Each **SeriesPaneView** is an object encapsulating how to draw a series on canvas given the data and scales (LC does similarly with pane views for each series type). E.g., `LineSeriesPaneView` will iterate visible points and draw line segments; `CandlestickPaneView` will draw candles, etc. They use the TimeScale and their PriceScale from the series to map data to pixels.
 - After series, we draw overlay elements: grid (vertical grid from TimeScale, horizontal from each PriceScale, clipped to pane), crosshair (if on this pane), and any primitives or markers (e.g., an arrow marker plugin draws on top).
 - Axes: We can draw axes on separate fixed canvases (not scrolling with content). The PriceScale API provides current scale to generate tick marks (like nice round numbers around min/max). We render axis labels for those ticks and the last price, and similarly time axis labels from TimeScale ticks.
- We take care to **only redraw what's necessary**: if only one pane changed, we need not redraw others; if only crosshair moved, we might only redraw the crosshair layer, not all series (to optimize, crosshair could be on a separate canvas).
- **Event Handling:** A central `InteractionManager` will handle DOM events (mouse, touch) on the chart container. It translates them to chart-space actions (e.g., a `mousemove` with position -> finds which pane and series (if any) is under cursor for hover highlights, updates crosshair position). It will

also manage state like “is panning”, “is dragging axis”, etc., and call the appropriate handlers. We design these handlers as methods on relevant components: e.g., `timeScale.startScaleDrag(atPixelX)` when user mouses down on time axis, `timeScale.scaleDragTo(newPixelX)` on move, and finally `endScaleDrag`. This encapsulation follows LC’s pattern of having separate logic in Axis widget classes, but we can simplify by keeping most logic in TimeScale/PriceScale classes themselves. **Hit-testing** for series (to detect which data point is nearest for crosshair snap or marker click) can be done via binary search on DataStore for X coordinate (since data is sorted by time) – very fast ($< \log n$). We’ll also maintain fast lookups of nearest index.

• **Data Flow Summary:** When new data arrives via `series.update()`:

- DataStore inserts the point (appends or replaces last).
- DataStore flags that series’ PaneView as invalid (needs redraw).
- PriceScale may update its stored min/max if the new point is outside current range (thus invalidating scale).
- RenderCoordinator is notified of these invalidations.
- On next frame, TimeScale might also extend (if a new bar at far right, it may not change visible range unless autoScroll is on and at right edge – if so, TimeScale will shift logical offset to keep right edge).
- Then the redraw steps occur, recalculating coordinates and painting new data.

• **Internal Module Interfaces:** We will define clean TypeScript interfaces for these internal boundaries:

- `IDataStore` with methods like `setData(array)`, `update(bar)`, `clear()`, `getNearestIndex(time)` etc.
- `ITimeScale` (or just class `TimeScale`) with methods `coordinateToTime(x)` and `timeToCoordinate(time)` and events `visibleRangeChanged` (for syncing, though we likely just handle internally).
- `IPriceScale` similar with `priceToCoordinate` and `coordinateToPrice`, plus methods to apply mode, margins, etc.
- `ISeriesPaneView` interface requiring `draw(ctx: CanvasRenderingContext2D, visibleRange: LogicalRange, precomputed?: any)` – each series type implements this. It might also have a `hitTest(x,y): HitTestResult` for interaction (to identify if a point or candle is under cursor).
- `IPane` that manages a collection of series views and primitives, with methods to add series (which then triggers adding a SeriesPaneView and linking to DataStore and PriceScale) and to draw all its content.
- `IChart` high-level interface (exposed API) which delegates to these managers.

Responsibility Partitioning & Rationale: This architecture ensures **loose coupling**: The data layer is unaware of pixel scale (so we can unit-test data parsing and updates independent of rendering). The scales are unaware of drawing specifics (they just compute transforms). Series drawing code deals only with how to draw a candlestick given a coordinate mapping – it doesn’t know about DOM events or outside world. This makes it easier to optimize each in isolation and swap components (for example, we could replace Canvas rendering with WebGL later by swapping out the PaneView implementations).

It mirrors LC's effective pattern (they had separate model objects for series and axes, with views subscribing to changes ¹⁴⁸). We go a step further by making a clear **RenderCoordinator** that can optimize draws and potentially handle offscreen rendering.

Data Flow Example: Suppose user zooms in: - InteractionManager catches wheel or pinch -> calls TimeScale.zoomAt(centerTime, scaleFactor). - TimeScale adjusts its logical range (barSpacing increases) and emits an internal event "scale changed". - Pane(s) get marked invalid (since X positions of all points change). - Price axes not affected by horizontal zoom, but if zooming in reveals fewer bars, autoscale might actually tighten vertical scale (less data visible). Our TimeScale will notify each series or DataStore of visible index range change (so they can compute their min/max in that range). PriceScales then recalc min/max and perhaps adjust (if autoScale). - Now RenderCoordinator draws: It sees series and axes invalid. It calls each Pane's draw. Each Pane calls each SeriesPaneView.draw with current scales. SeriesPaneView asks DataStore for visible points (between TimeScale.visibleRange.from/to). DataStore can efficiently slice the data array to just needed points (or even decimate if too many points per pixel – handled by conflation logic). - Canvas is updated with new drawn shapes. Axes are updated with new labels (e.g., time axis labels might now show hours:minutes if zoomed enough to see intraday). - The user perceives a smooth zoom with all content updating consistently.

Boundary of Modules: - The **Interfaces between modules** will include: - DataStore <-> SeriesPaneView: SeriesPaneView calls DataStore for data access. DataStore does not call view (no direct coupling; it triggers via events/invalidation). - DataStore <-> PriceScale: DataStore (or rather Series) registers itself with a PriceScale. PriceScale provides a callback or method for DataStore to report "my data range changed" so PriceScale recalcs. Conversely, PriceScale exposes to DataStore what the current visible range is (so DataStore could optimize e.g. purge data outside far ranges, though actual purge is DataProvider's job). - Pane <-> PriceScale: Pane uses PriceScale for axis drawing and for converting series values. PriceScale doesn't know about Pane details, it just holds state. - Chart <-> Panes: Chart orchestrates multi-pane layout, and on overall events like resize or new data, it delegates to each Pane accordingly. - Plugin interface: e.g., a plugin wants to draw an overlay (like watermark). We provide either an `IPanePrimitive` interface where plugin supplies a `draw(ctx, paneInfo)` method. The Chart will call these at appropriate time (either before series if it's an underlay, or after if overlay). - We also have an extension point for **custom series** (which is a plugin type but tied to data). For that, we mimic LC's approach with `ICustomSeriesPaneView` ⁷¹ and possibly `ICustomSeriesRenderer` that plugin devs implement (with methods to convert their custom data to "bar" info for crosshair, etc.). The Chart, when drawing custom series, calls their renderer with the current context and scale – enabling any custom drawing (could be non-standard chart like a profile, etc.).

Performance Considerations in Architecture: By separating data and view, we can offload heavy computations. For example, computing the visible data slice and doing conflation can be done in the DataStore module, possibly even in a Web Worker if needed, since it doesn't touch the DOM. The RenderCoordinator can be configured to let certain work happen off the main thread (like precomputing conflated averages).

Parallel with LC internals: LC's internal architecture has Model classes (PriceScale, TimeScale, Series, etc.) and separate Views (each series has a PaneView, etc.), and they communicate via an internal pipeline. We adopt the same proven pattern ¹⁴⁸, which ensures our system will be familiar to someone reading LC's source. We add improvements like a clearer Plugin API (for markers & drawings) and a centralized rendering scheduler to better handle rapid updates.

Module/File Structure (Tentative):

```
src/  
  Chart.ts      (Chart class - manages overall chart, API methods)  
  Pane.ts       (Pane class - one per pane)  
  Series.ts     (Series base class - holds DataStore, references PriceScale)  
  SeriesPaneView.ts (abstract; each series type subclass provides draw logic)  
  SeriesTypes/  
    LineSeries.ts, CandlestickSeries.ts, etc. (specific data processing or  
defaults)  
    LinePaneView.ts, CandlestickPaneView.ts, ... (drawing logic per type)  
  Scales/  
    TimeScale.ts, PriceScale.ts (scale classes)  
  Data/  
    DataStore.ts (generic or per series; could have subtypes for OHLC vs  
single-value)  
    DataProvider.ts (handles on-demand loading, chunk mgmt)  
  Interaction/  
    InteractionManager.ts (handles events, crosshair logic)  
    HitTest.ts (utilities for point proximity checks)  
  Plugins/  
    SeriesMarkerPlugin.ts, UpDownMarkerPlugin.ts, WatermarkPlugin.ts, etc.  
(implement IPanePrimitive or ISeriesPrimitive)  
    RenderCoordinator.ts (schedules and triggers draws, handles invalidation  
flags)  
    Utils/ (misc utilities: formatting (e.g. DateTimeFormatter), color parsing for  
P3, etc.)
```

Each file encapsulates one component. For example, `TimeScale.ts` manages tick mark generation (with helper functions possibly for nice time intervals), and `PriceScale.ts` might include static helpers for log scale calculations (ensuring numerical stability).

Why this structure: It groups related responsibilities, making it easy to maintain and test. For instance, `SeriesPaneView` and its subclasses can be unit-tested with dummy data and a mock scale (ensuring drawing commands output expected coordinates). The separation also aids tree-shaking: if an application never uses `CandlestickSeries`, that module can be dropped (similar to LC v5 making certain features plugin-based for size ¹⁵⁹).

In our improved design, even core features like series markers and watermarks are modular (LC moved them to plugins in v5 ¹⁵⁹). We maintain that: our core modules will not depend on plugin modules – they'll register via an extension mechanism. E.g., the Chart might have a `registerPlugin(pluginInstance)` method that attaches hooks (like `onRender`) or adds a `PanePrimitive`. This way, optional features don't bloat the core logic.

Data flow diagram: (Pseudo-illustration)

```

User Input (mouse/touch) --> InteractionManager --> Chart/Pane/Scale update
(like change range or crosshair pos) --> invalidate --> RenderCoordinator -->
SeriesPaneView.draw --> Canvas
Data Update (API) --> Series/DataStore update --> PriceScale (new range) &
TimeScale (maybe new bar) --> invalidate --> RenderCoordinator (next frame) -->
redraw

```

All arrows are event or method calls in code.

Overall, this architecture ensures **fast, synchronous UI updates** for interactions, and splits heavy computations (like finding visible points or recalculating scales) into manageable pieces. By following these internal boundaries, we can confidently extend or optimize parts (for example, swapping the Canvas drawing with WebGL just means writing new PaneView classes without changing Data or Interaction logic). The modular design also will facilitate **unit testing** each piece: e.g., feed synthetic data into PriceScale, test that coordinate conversion matches expected values for known modes (linear/log etc.), or simulate zoom events to check that TimeScale and Pane produce correct new range and call redraw appropriately.

4. Rendering & Performance Plan (60fps-first)

Our foremost goal is a sustained **60 FPS** render even under heavy loads (many data points, frequent updates, or multiple overlaid visuals). To achieve this, we design a careful rendering invalidation system, optimize canvas usage, and incorporate downsampling strategies for large datasets.

Animation Loop & Invalidation: We use `requestAnimationFrame` (rAF) to coordinate redraws. Rather than redrawing immediately on every data or mouse event, we mark components as “dirty” and schedule a rAF. The **RenderCoordinator** collects these invalidation flags: - If only the crosshair moved, we can redraw just the crosshair layer (small optimization: keep crosshair on a separate overlay canvas). Similarly, if only a single series' data changed and it doesn't affect others (and axes), we could redraw only that pane's content and perhaps just recompute that PriceScale. - On rAF, the coordinator can batch multiple updates. For example, if 50 new points stream in one by one in a single frame, they all mark the series dirty but we redraw once after processing all, using the final state. - We measure each frame's render time (using `performance.now()`) to adapt if needed: e.g., if rendering all series every frame is too slow, we might skip some frame updates under extremely heavy load (dropping frames is a last resort, but better than jank – more on monitoring below).

Canvas Layering: We plan to use multiple canvases: - **Main Plot Canvas per Pane:** Draws series and grid. - **Overlay Canvas per Pane:** Draws crosshair, markers, and possibly axes (axes could also be their own canvas to allow easy CSS styling or fixed position). - **Separate Axis Canvases:** One for each visible price axis and one for the time axis. This simplifies not having to clear and redraw axes on every pan (they only change when scale changes or crosshair moves for the highlighted label).

Using layers means we can clear and redraw crosshair 60 times per second (when mouse moves) without redrawing series. Series canvases only redraw on data or zoom events (which are less frequent). This drastically reduces work on typical interactions (e.g., moving crosshair is just overlay canvas update, which is cheap).

We keep canvases sized appropriately (with devicePixelRatio accounted). Clearing a canvas is $O(\text{width} \times \text{height})$ – we minimize canvas size per layer. For example, the axis canvas width might be ~50px, height full chart – clearing that is minor compared to a full chart canvas. The main series canvas still could be large (e.g., 1000x500 px), but we only redraw it when needed.

Drawing Optimization: - We will use **Path2D** for batch drawing where possible (e.g., building a Path2D for an entire line series and then stroking it once). Path2D can sometimes be cached – however, since it depends on scale, caching across frames is limited. But if zoom is constant and only data is streaming at right edge, we could append to an existing Path2D rather than redraw entire line – this might be micro-optimized by keeping track of last drawn segment (though need caution to redraw if something invalidated previous segments). - For candlesticks/bars: drawing thousands of individual rectangles can be heavy. We can reduce overhead by pre-calculating points to pixels in arrays and then iterating efficiently, or even using `CanvasRenderingContext2D.drawImage` sprites for repetitive shapes (not likely beneficial for varying heights). But our main speed-up will be to **skip drawing elements that collapse to <1px**: e.g., if many bars are so thin at far zoom, conflation will merge them or we simply skip subpixel-width bars to avoid useless work (LC conflation handles this elegantly by merging points when bar spacing $< 0.5\text{px}$ ¹⁶⁰). - Text rendering is a known slow path. We minimize text draws: only axes and perhaps last price labels require `fillText` operations. We ensure we do not draw axis labels every frame unless needed (if no scale change, no need to re-render axis labels except perhaps highlight of last label on crosshair move). For crosshair price label which updates each movement, that is one text draw per frame – which is fine.

Downsampling & LOD: Data conflation (Level of Detail reduction) is critical for performance with huge data: - We will implement **automatic LOD selection**: if the time scale is so zoomed out that multiple data points map to one pixel column, we replace them with a conflated point. Our conflation approach matches LC v5.1: when bar spacing $< 0.5\text{px}$, combine points ¹⁶⁰. We'll default enable this (LC requires opt-in `enableConflation`), but we can choose to auto-enable or at least encourage it for large data). When enabled, our `DataStore` will, for each rendered index interval, aggregate blocks of points. For line/area we might take the average or perhaps min/max if visual envelope is needed (LC likely uses something like min/max or last for line? Actually, conflation presumably uses average for smoothing ¹⁶¹). For OHLC, we can aggregate open of first, close of last, and min low and max high in that pixel span (so candlesticks conflated become a wider “candle” showing range). - We provide `conflationThresholdFactor` to tune smoothness vs detail (just like LC) ¹⁶². E.g., factor 2.0 means conflation kicks in earlier, yielding smoother chart at slight expense of detail ¹⁶². - Also, for very large number of series (if many series stacked), we ensure only visible series are actively drawn. If a series is set `.visible=false`, we skip all its calc and drawing (LC does this too). - **Memory Strategy for Data:** We avoid holding full raw data in memory when not needed. For instance, if a series has 2 million points but the chart can only display 100k distinct X positions, storing all 2M in active memory might be heavy (though not terrible in JS if just numbers, but ~16MB for 2M objects possibly). With chunking, we can keep only relevant chunks in memory and maybe store others compressed (if historical data). - The `DataProvider` will unload off-screen data: e.g., if user is zoomed into recent 1% of data, we might unload the far history temporarily (assuming it can be fetched again from some source if needed). This is an app-level decision; our architecture allows plugging a fetch mechanism. - We could also compress data points (for instance, storing only float32 instead of JS objects). Possibly we allocate typed arrays for times and values. This has performance upsides (fewer GC objects) and memory savings (32-bit vs 64-bit double). We must be careful with precision, but for charting ~7 decimal precision is fine for time stamps and price typically. We plan to implement an option to use typed arrays internally for series data.

Garbage Generation: We strive for zero-allocation during continuous interactions: - We'll reuse objects where possible (e.g., a single crosshair `MouseEventParams` object that we update and pass to subscribers, instead of new object on each move). - In drawing, avoid creating new arrays every frame. Preallocate arrays for coordinates of each series of size = number of points (update them as needed). Or use static buffers. - Avoid closures in hot loops (use for-of or indexed loops rather than `.forEach` with lambda, etc.). Modern JS engines handle these well, but we'll be mindful. - The conflation computation might allocate a new array of conflated points each time zoom changes significantly - we can cache conflated results at certain zoom levels. LC introduced `precomputeConflationOnInit` and background tasks ¹⁶³ - we can do similar: e.g., precompute coarse representations (maybe keep downsampled data for various factors in memory). - Use object pooling if necessary for frequent small objects (like tick mark objects each frame; but better to just reuse tick structures and update values/text).

Frame Budget & Scheduling: We will instrument the rendering to measure time: - If a draw of a certain series or pane consistently takes too long (e.g. >16ms), we log a warning. Our benchmarking harness (discussed below) will help catch this. - For extremely high-frequency updates (10k points/sec), we likely cannot draw every single update as a separate frame. But because the data is streaming, often those 10k points might be a batch (like 10k points added per second ~ 166 points per frame at 60fps). Our system can handle 166 new points if drawn as one polyline extension (very feasible). If it were 10k *per frame*, we would rely on conflation to reduce overhead (most likely the chart is zoomed out in such case, conflating them). - We consider using **Web Workers** for offloading heavy work: For example, if computing conflation or searching big data sets, a worker could do it and then main thread just draws. OffscreenCanvas could allow worker to even do the drawing on a bitmap. However, OffscreenCanvas with 2D context in worker is supported in some browsers - as an *optional enhancement*, we design our Renderer so it could run in a worker thread with an OffscreenCanvas (if available). But initially, we'll stick to main thread rendering and offload only computations (like data parsing). - **Text rendering strategy:** Canvas text drawing is not easily offloaded, but we can mitigate by limiting how often we redraw text. For price axes, the labels only change when price scale changes or crosshair moves. Crosshair label is redrawn often, but it's just one text - negligible. The axis tick labels change only on zoom or new price extremes - so infrequent. If needed, we can also render common labels to a static canvas and reuse (like an atlas of number glyphs) but likely overkill.

Pointer Event Throttling: We ensure that if `mousemove` events fire at 120Hz (some high-end mice or touch generate lots of events), we do not process them faster than rAF. Essentially, our crosshair update is already tied to rAF by invalidation, so it naturally throttles: multiple mouse events within one frame just set one flag. If needed, we could explicitly ignore events if one is in flight, but rAF gating suffices.

Memory Management & Limits: We keep an eye on memory: - 10 series × 2M points might be ~20 million points. If each point is (time, value) as two 64-bit numbers, that's 16 bytes each, ~320 million bytes (~305MB) raw, plus JS object overhead if not using typed arrays (which could blow it up significantly). That's too high for the browser. Clearly, we need strategies: - Use `Float32Array` or `Float64Array` for values and possibly a `Uint32` or `Float64` for timestamps (Unix timestamp up to seconds fits in 32-bit; milliseconds needs 64-bit). That would cut overhead drastically (no per-point object, just contiguous array). - Also, not all 20M need to reside in memory simultaneously. The chunking approach means maybe only, say, 2M are ever loaded at once (others on server or compressed). If data is static and local, perhaps we store it in a compressed form and only expand for current view. - We can integrate a *simplified in-memory database* like structure: index by time, allow binary search, and page in/out. But that might be beyond initial scope. For now, we plan that if extremely large dataset scenario (Scenario D in user assumptions), the application will supply data in

chunks and maybe remove older ones as needed using an API (like `series.setData` for initial and `series.update` for appends, and maybe an API to drop earliest points if needed). Our architecture will support removal (we have `series.pop(n)` as in LC 5.0.9 ¹⁶⁴ to remove N points from end or start). - Ensure no memory leaks: when series are removed or data is cleared, we null references so GC can reclaim it. If OffscreenCanvas is used and then disposed, properly terminate it.

Large-Data (LOD) Rules: - If total points > some threshold (e.g. >50k), enable conflation automatically (unless user explicitly opts out). This ensures that zoomed out views are fast. - We can implement multi-stage downsampling: e.g., first do a simple skip (if one pixel covers 100 points, we might take every 10th for a quick draw if conflation is off – though that can alias; conflation (averaging) is better visually). - If memory is a concern, we might not store all raw points in JS objects *and* also conflated points – instead, compute conflation on the fly for current view. LC's `precomputeConflationOnInit` option ¹⁶⁵ suggests they can precompute multi-resolution. We could do something similar in a Web Worker: build reduced sets at factors 2x,4x, etc. We'll include this as an optional performance booster (with trade-off: memory vs CPU). - Also consider skipping frames during continuous zoom: if the user uses mousewheel to zoom continuously, rather than recalculating 30 new conflations as they zoom, we might temporarily degrade fidelity (skip some intermediate zoom frames) to keep it smooth. But given modern hardware, recalcing at 60fps for typical data might be okay – we will measure this scenario.

FPS and Latency Metrics: We will build into dev mode a tiny overlay or console logs that report: - Last frame render time (ms). - The current FPS (smoothed). - Event latency: e.g., time from last data update to it being drawn. - Memory usage estimate: e.g., number of data points in memory (since true memory from JS is hard to measure reliably, but we can log counts).

We also add internal markers (using `performance.mark`) around key phases (like “layout calculation”, “canvas draw”) so in browser devtools one can see where time is spent.

Interactions per second: Crosshair moves can be very frequent (user wiggles mouse). Our design ensures crosshair rendering is lightweight – a single line draw, and updating two labels. We expect that to easily handle hundreds of events per second without dropping frames.

For extremely rapid tick updates (10k/sec), we rely on not drawing each tick individually. If needed, we consider **coalescing updates**: e.g., use `requestAnimationFrame` scheduling such that if multiple updates arrive, only latest is drawn. Our API could provide `series.updateMany(dataArray)` to append a batch in one go (the user scenarios mention `appendBatch` method – we will have that, which reduces overhead by merging many updates into one draw call).

Memory of old points: If DataProvider unloads old data, we ensure visual continuity by possibly keeping a minimal summary (maybe last point of previous chunk to connect line smoothly). But if it's removed entirely, we must handle that gracefully (no weird gaps aside from intended). We will allow “whitespace” data points to explicitly mark gaps (as LC does with `WhitespaceData` objects ¹⁶⁶). Removing data via pop might implicitly create a gap if one deletes an intermediate region – but that's uncommon. Usually pop is used to drop oldest (left-most) data which just moves the start.

Target Bundle & Performance: We aim for a **bundle ≤ 60KB gzipped** (per user requirement). Our design of splitting plugins (markers, etc.) out of core helps. Also, using modern ES2020 syntax (as LC does ¹⁶⁷)

allows terser output. We monitor build size per module and will tree-shake unused series types. This means if an app only uses line and candlestick, the others won't add weight.

In terms of runtime performance: - We expect to handle Scenario D (10 series × 2M points). With conflation and not all in memory at once, we're effectively drawing at most ~ `chartWidth` points per series on screen (e.g., ~1000 points if 1000px wide) – which is fine. Memory wise, 20M points might be too much to keep – but maybe the app wouldn't load all 20M simultaneously. If needed, a 64-bit timestamp + 32-bit price for 20M points = $\sim 20M * 12 \text{ bytes} \approx 240MB$, plus overhead. With chunking we could keep, say, 100k points per series in memory (1M total across 10 series) which is reasonable, and load more on scroll. - Scenario E (10k points/sec streaming): Our update pipeline can easily append 10k points every second if it's batched (that's ~166 points per frame). Even if done one-by-one (worst case), each update is $O(1)$ insertion and drawing the last segment. The bigger cost is shifting the time axis – but our time axis simply moves the logical offset or extends scale, which is trivial. We must ensure GC doesn't choke on 10k new objects/sec – using typed arrays for data here massively helps (push in a typed array is just setting elements, not creating new JS objects). Also, if 10k/sec goes on for minutes, that's a lot of data accumulation – again chunking or periodic decimation might be needed to avoid runaway memory. Possibly we could drop intermediate points if they come too fast relative to pixel (like if 100 points fall in the same pixel between rAF frames, many are redundant for visual – but user might expect them in memory for later scroll, so we probably keep them, just don't draw all).

Measurement Plan: We'll integrate a lightweight performance monitor: - Use `performance.now()` in dev builds around critical functions (like each series draw). - Possibly allow an **adaptive detail** mode: if a frame took > 16ms, output a console message like "Chart rendering is slow (X ms); consider enabling conflation or reducing data." For automated tests, we'll have thresholds (see Benchmark Harness).

Threading & OffscreenCanvas: As an optional enhancement, we will implement an OffscreenCanvas path for heavy drawing: - Move the series drawing for each pane to an OffscreenCanvas in a Worker. The worker can receive data updates and draw commands, and then transfer the bitmap to the main canvas. This can keep main thread free for UI. - However, not all browsers support OffscreenCanvas for 2D in Worker (Chrome does; Safari might not). We design our system to easily toggle this: basically, if OffscreenCanvas is available and `useWorker=true` option, then on init we create workers for each pane or one worker for all rendering, transfer OffscreenCanvas to it, and route draw calls to worker. The worker then posts back when a frame is ready, and main just blits (if needed). - For simplicity, we might use one worker for all panes (to avoid multi-thread sync issues). - This is advanced and might not be in initial release but is part of our roadmap for "performance-first".

Summary: By combining *intelligent invalidation*, *LOD data reduction*, *efficient canvas layering*, and *careful JavaScript practices*, our chart aims to remain buttery smooth under all typical usage. We leverage lessons from TradingView LC (which is known to handle a lot) and push further with background tasks and memory optimizations. We will continuously profile the system with large data and on various devices (especially mobile, which can be the limiting factor in performance) to ensure our frame times stay within budget. The **"what to measure"** specifically: frame render time distribution (mostly under 16ms, 98th percentile maybe < 20ms), input event handling delay (from user event to chart reaction < 50ms ideally), memory usage vs data volume (should be roughly linear and acceptable for given scenarios). These metrics feed into our Benchmark & Regression tests described next.

5. API Proposal (TypeScript)

We will design a **TypeScript API** closely aligned with Lightweight Charts for easy adoption, but with added safety and clarity. Key differences include stricter typing (to catch misuse early) and more extensibility for plugins. Below are the key interfaces and classes:

```
// Core chart creation functions:
function createChart(container: HTMLElement, options?:
DeepPartial<ChartOptions>): IChartApi;
function createChartPane(container: HTMLElement, options?:
DeepPartial<ChartOptions>): IChartApi;
// (alias for createChart)

function createPriceChart(container: HTMLElement, options?:
DeepPartial<PriceChartOptions>): IChartApi;
// price-based horizontal axis

function createYieldCurveChart(container: HTMLElement, options?:
DeepPartial<YieldCurveChartOptions>): IYieldCurveChartApi;
// yield curve specialized chart

// Main Chart interface
interface IChartApi {
  addPane(options?: PaneOptions): IPaneApi;
  removePane(pane: IPaneApi): void;
  addSeries<T extends SeriesType>(seriesType: T, options?:
SeriesOptionsMap[T]): ISeriesApi<T>;
  addHistogram(options?: HistogramSeriesOptions): ISeriesApi<'Histogram'>;
  // ... (Convenience overloads for each series type, e.g. addLine,
addCandlestick, etc.)

  timeScale(): ITimeScaleApi;
  priceScale(position?: 'left'|'right'|string):
IPriceScaleApi; // default to 'right'
  // (If custom id provided that doesn't exist, perhaps returns undefined or
creates new invisible scale)

  applyOptions(options: DeepPartial<ChartOptions>): void;
  options(): Readonly<ChartOptions>;

  subscribeClick(handler: (param: MouseEventParams) => void): void;
  unsubscribeClick(handler: (param: MouseEventParams) => void): void;
  subscribeCrosshairMove(handler: (param: MouseEventParams) => void): void;
  unsubscribeCrosshairMove(handler: (param: MouseEventParams) => void): void;

  // Lifecycle
```

```

        resize(width: number, height: number, forceRepaint?: boolean): void;
        remove(): void;
    }

    // Pane API (returned from chart.addPane)
    interface IPaneApi {
        addSeries<T extends SeriesType>(seriesType: T, options?:
SeriesOptionsMap[T]): ISeriesApi<T>;
        addCustomSeries<TData, TOpts extends CustomSeriesOptions>(
            renderer: ICustomSeriesRenderer<TData, TOpts>, options?:
DeepPartial<TOpts & SeriesOptionsCommon>
        ): ISeriesApi<"Custom", TData>;
        removeSeries(series: ISeriesApi<any>): void;

        priceScale(id?: string): IPriceScaleApi; // default id returns default
scale for pane
        applyOptions(options: DeepPartial<PaneOptions>): void;

        setHeight(height: number): void;
        getHeight(): number;
        setStretchFactor(factor: number): void;
        getStretchFactor(): number;
        paneIndex(): number;
        moveTo(index: number): void; // reorder pane in chart
        // ... preserveEmpty, etc.
    }

    // Series API (generic for all series types)
    interface ISeriesApi<T extends SeriesType, TData = SeriesDataItemTypeMap[T]> {
        setData(data: readonly TData[]): void;
        update(bar: TData): void;
        updateMany(bars: readonly TData[]): void; // (our addition for batch
updates)
        setMarkers(markers: SeriesMarker<T>[]):
void; // uses plugin under the hood if needed
        applyOptions(options: DeepPartial<SeriesOptionsMap[T]>): void;
        options(): Readonly<SeriesOptionsMap[T] & SeriesOptionsCommon>;

        priceScale(): IPriceScaleApi; // which scale this series is bound to
        seriesType(): T;
        dataByIndex(index: TimePointIndex): TData |
null; // allow access to a point by logical index if needed

        // New event:
        subscribeDataChanged(handler: DataChangedHandler<TData>): void;
        unsubscribeDataChanged(handler: DataChangedHandler<TData>): void;
    }

```

```

// Price Scale API
interface IPriceScaleApi {
    applyOptions(options: DeepPartial<PriceScaleOptions>): void;
    options(): Readonly<PriceScaleOptions>;
    // Methods to convert values (provided after chart is created, when scale is
    ready):
    priceToCoordinate(price: number): number | null;
    coordinateToPrice(y: number): number | null;
}

// Time Scale API
interface ITimeScaleApi {
    applyOptions(options: DeepPartial<TimeScaleOptions>): void;
    options(): Readonly<TimeScaleOptions>;

    scrollToPosition(position: number, animate?: boolean): void;
    scrollToRealTime(): void;
    getVisibleRange(): TimeRange | null;
    setVisibleRange(range: TimeRange): void;
    resetTimeScale(): void; // e.g., undo manual scroll/zoom to default
    fitContent(): void;     // show all data
    // coordinate converters:
    timeToCoordinate(time: Time): number | null;
    coordinateToTime(x: number): Time | null;

    subscribeVisibleRangeChange(handler: TimeRangeChangeEventHandler): void;
    unsubscribeVisibleRangeChange(handler: TimeRangeChangeEventHandler): void;
}

// Options Interfaces (structures defining configurable settings)
interface ChartOptions {
    layout: LayoutOptions;
    crosshair: CrosshairOptions;
    grid: GridOptions;
    handleScroll: HandleScrollOptions | boolean;
    handleScale: HandleScaleOptions | boolean;
    kineticScroll: KineticScrollOptions;
    timeScale: TimeScaleOptions;
    // possibly: rightPriceScale, leftPriceScale defaults, etc.
    localization: LocalizationOptions;
    panes: LayoutPanesOptions; // global pane settings (separator colors, etc.)
}
interface PriceChartOptions extends ChartOptions {
    // Could include specific things for price-based charts if any (likely none
    beyond normal)
}
interface YieldCurveChartOptions extends ChartOptions {
    yieldCurve: {

```

```

        baseResolution: number; // e.g. 1 (month)
        startTimeRange: number; // e.g. 3 (months) for initial spacing
        minimumTimeRange: number;
    };
}

interface SeriesOptionsCommon {
    priceScaleId?: string;
    visible: boolean;
    title: string;
    lastValueVisible: boolean;
    priceLineVisible: boolean;
    priceLineSource: PriceLineSource;
    priceLineWidth: LineWidth;
    priceLineColor: string;
    priceLineStyle: LineStyle;
    baseLineVisible: boolean;
    baseLineColor: string;
    baseLineWidth: LineWidth;
    baseLineStyle: LineStyle;
    autoscaleInfoProvider?: AutoscaleInfoProvider;
    // etc., plus:
    lastPriceAnimation: LastPriceAnimationMode;
}

// Series-specific options:
interface LineSeriesOptions { color: string; lineWidth: LineWidth; lineStyle:
LineStyle; lineType: LineType;
    crosshairMarkerVisible: boolean; crosshairMarkerRadius: number;
crosshairMarkerBorderColor: string; crosshairMarkerBackgroundColor: string;
crosshairMarkerBorderWidth: number;
    pointMarkersVisible: boolean; pointMarkersRadius?: number;
}

// ... similar interfaces for CandlestickSeriesOptions, etc. (upColor,
downColor, borderVisible, etc.)

// Data types:
type Time = UTCTimestamp | BusinessDay | string;
type UTCTimestamp = number; // seconds since epoch
interface BusinessDay { year: number; month: number; day: number; }

type SeriesType = "Line" | "Area" | "Bar" | "Candlestick" | "Histogram" |
"Baseline" | "Custom";
type SeriesDataItemTypeMap = {
    Line: SingleValueData;
    Area: SingleValueData;
    Bar: OhlcData;
    Candlestick: OhlcData;
    Histogram: HistogramData;
}

```

```

    Baseline: SingleValueData;
    Custom: any; // custom series data is user-defined
};
interface SingleValueData { time: Time; value: number; }
interface OhlcData { time: Time; open: number; high: number; low: number;
close: number; }
interface HistogramData { time: Time; value: number; color?: string; }

interface SeriesMarker<T extends SeriesType = SeriesType> {
    time: Time;
    position: "aboveBar" | "belowBar" | "inBar"; // placement relative to bar
    shape: MarkerShape; // e.g. "circle" | "arrowUp" | "arrowDown"
    color: string;
    text?: string;
    size?: number;
}
// ... etc.

// Event parameter types:
interface MouseEventParams {
    time?: Time;
    point?: Point; // x,y coords on chart
    paneIndex?: number;
    seriesData: Map<ISeriesApi<any>, { value: number | OhlcData; index:
TimePointIndex }>; // value at crosshair for each series
    hoveredSeries?: ISeriesApi<any>;
    hoveredMarkerId?: string; // if we assign ids to markers
}

interface DataChangedHandler<TData> {
    (series: ISeriesApi<any>, data: TData[], meta: DataUpdateType): void;
}
// DataUpdateType could be "full-set" | "append" | "update-last" | "remove",
etc., to indicate what changed.

```

(Note: The above is a representative snippet; the full API will include all enumerations and option interfaces. We have ensured to preserve names and semantics from LC v5.1 for familiarity, while adding types and a few convenience methods.)

Notable API improvements:

- We introduced **batch update** (`updateMany`) on series for efficiency when streaming multiple points, so users don't have to loop and cause multiple re-renders.
- The `subscribeDataChanged` event on series provides a unified way to listen for any data modifications on that series ¹⁵⁴, useful for updating legends or debugging. (LC has `DataChangedScope` concept internally; we expose a simplified event).

- We have **explicit types for series options** (LC uses generics but we make them more concrete in docs). For example, `LineSeriesOptions` vs `CandlestickSeriesOptions` are separate, and the user gets compile-time error if passing candlestick-specific option to a line series.
- **Stricter Time type:** We unify `Time` as union of UTC timestamp (number, assumed seconds) and `BusinessDay` and ISO string ¹²². Our API will enforce correct type shape: e.g., if an object with year/month/day is passed, it must match `BusinessDay` interface (LC accepts any object with those keys; we do too, but typing helps).
- **DeepPartial** utility is used to allow passing partial options objects (so users can supply only what they want to change).
- The API supports multiple panes and custom series natively (LC v5 does, but through generic `addSeries` with definitions; we simplified via method overloads and explicit `IPaneApi`).
- We unify creation functions naming: `createChart`, `createPriceChart`, `createYieldCurveChart` for clarity (LC uses `createOptionsChart` for price-based, which is a bit non-obvious naming ¹³; we choose `createPriceChart`).
- **Plugin API:** We hinted custom series via `addCustomSeries(renderer, options)` where `renderer` implements an interface to draw itself. For pane overlay plugins (like watermark), we will expose methods on chart or pane such as:

```
chart.addWatermark(options: TextWatermarkOptions): IWatermarkPlugin;
pane.attachPrimitive(primitive: IPanePrimitive): void;
```

This allows advanced usage without cluttering core API for basic users. For example, `createSeriesMarkers(series, markers, options)` in LC is a function ¹⁶⁸ – we could integrate that into `series.setMarkers` as done above, which internally uses the plugin but hides complexity from user.

- **Type Safety & Defaults:** We ensure that if a series requires a certain structure of data, the TS type enforces it (e.g., `ISeriesApi<'Candlestick'>` will only accept `OhlcData` in `setData/update`). Similarly, options are typed so passing a misspelled option will error. Defaults for each option are documented (as seen from LC docs with default values ¹⁶⁹ ⁸⁶).

Event handling differences: We follow LC for click and crosshair events naming, so existing code can adapt easily. The `MouseEventParams` we propose contains a Map of series values at that point (LC provides `seriesPrices` record in their param, but only for series on the default price scale in older version; in v5 it might be more flexible). We choose Map of series to value for extensibility – multiple series including custom ones can be included. This is more type-safe than LC's approach of a generic object with series ID keys.

Backward compatibility: This API is largely compatible with LC's (a developer familiar with LC can quickly use ours). Key differences like `addSeries` signature (we allow optional pane index) mimic LC's `IChartApiBase.addSeries` which had an optional `paneIndex` param in v5 ¹⁷⁰. We include it in `IPaneApi.addSeries` instead (more logical – you add series to a specific pane via that pane's API).

Examples (to illustrate usage):

```

const chart = createChart(div, {
  width: 800, height: 400,
  layout: { background: { type: 'solid', color: '#ffffff' }, textColor:
    'black' }
});
const mainPane = chart.addPane(); // default pane already exists, addPane adds
second one
const candlestickSeries = chart.addSeries('Candlestick', { upColor: 'green',
downColor: 'red', wickDownColor: 'red', wickUpColor: 'green' });
candlestickSeries.setData(candleData);
const volumeSeries = mainPane.addSeries('Histogram', { color: '#26a69a' });
volumeSeries.setData(volumeData);

// Price scale adjustments:
chart.priceScale('right').applyOptions({ mode: PriceScaleMode.Logarithmic });
chart.timeScale().fitContent();

// Crosshair customization:
chart.applyOptions({ crosshair: { mode: CrosshairMode.MagnetOHLC, vertLine: {
color: '#555', width: 1, style: LineStyle.Solid } } });

// Subscribe to events:
chart.subscribeCrosshairMove(param => {
  if (param.time) {
    console.log("Crosshair at time", param.time, "prices:",
[...param.seriesData.entries()].map(([series, val]) => val));
  } else {
    console.log("Crosshair left chart");
  }
});

```

This snippet would be fully type-checked, ensuring correct option keys and data shapes.

In summary, our API is **minimal but extensible**: we cover all core capabilities of LC (series types, multi-pane, etc.) while providing hooks for custom behavior and plugins. We've prioritized naming consistency and type safety, so developers can avoid common errors (like using wrong series type data). All major behaviors (scroll, zoom, scale modes, etc.) are exposed through these APIs. For advanced usage (like creating a completely new chart type or custom scale behavior), we have the `createChartEx` and custom series/primitive interfaces which advanced devs can use to plug in new functionality without modifying the core.

6. Benchmark & Regression Harness

To ensure our charting system is truly “better than” the existing one, we will build a comprehensive **benchmark and regression testing harness**. This harness will simulate real use cases (scenarios A-E mentioned) and measure performance metrics, as well as verify visual correctness against known outputs.

Datasets & Scenarios:

We define several test scenarios with synthetic and real-world data: - **SCENARIO_A (Small static)**: ~1K points line series, no streaming. Used to verify basic rendering correctness (image diff) and baseline performance (should be trivially fast). - **SCENARIO_B (Large static)**: 1M points line series (sine wave or random walk). We test initial load time and zoom/pan performance with conflation on vs off. This measures our data structure efficiency and LOD. - **SCENARIO_C (Multi-series static)**: 5 series * 100k points each (mixed types: candlestick, line, area, etc.). Check multi-scale interactions (one series on left scale, others on right or overlay) and ensure no performance cliff. - **SCENARIO_D (Huge historical)**: 10 series * 2M points each as per assumption. Data can be generated as e.g. 10 years of minute bars for 10 stocks (so candlestick series for each). The harness will attempt operations: zoom all the way out (ensuring conflation engages), scroll through the entire range (observing chunk loading if implemented), and zoom into a small range. We log FPS during these actions. We also monitor memory: ensure it stays below an acceptable threshold (likely we expect $\leq$ ~300MB usage in worst case of this scenario, and ideally much less if chunking unloads old data). - **SCENARIO_E (Real-time stream)**: Simulate incoming ticks: e.g. 1 series receives 10,000 points per second for 5 seconds (50k total points). This can be modeled as a loop that calls `series.update` in rapid succession or uses `updateMany` in chunks of 1000 every 0.1s. We measure frame rate and latency (does chart keep up in real-time?). We also check memory (should not balloon uncontrollably with short-term high throughput).

Performance Metrics: We will gather: - **FPS (Frames Per Second)**: using `requestAnimationFrame` timestamp deltas. For each scenario, we calculate average and 1% low FPS. - **Frame Time Distribution**: The harness can record each frame's draw time (via performance marks in our library) to see if any frames exceed 16ms. - **Memory usage**: not easily directly from JS, but we can use Chrome's performance memory timeline or simply infer via object counts. Alternatively, our harness can run in Node with a headless canvas to inspect memory (though headless environment memory may differ). We might embed a hidden method to output current memory stats (like total data points in memory, maybe using `performance.memory` if available in browsers). - **Input latency**: We can measure time from an input event to the chart state change. E.g., programmatically dispatch a mousemove and then check how quickly crosshair moved (this is tricky to automate in real browser, but we can instrument our code: mark time at event handler entry and when we finish drawing crosshair, then log the delta). We aim for < 20ms for crosshair moves.

Visual Regression: We will use an automated screenshot comparison for known chart states: For example, we know what a simple candlestick chart should look like. We will generate an image using our library and compare pixel-by-pixel to either: - A reference image (perhaps generated by LC for the same data and options). - Or a previously saved image from our library (to catch changes over development).

We will set up a small suite: - Render a chart in various configurations (line chart with known random seed data, candlestick with set values, with and without log scale, etc.) in a headless browser (using Puppeteer or Playwright). - Capture PNG and compare to reference (with a tolerance for anti-aliasing differences if needed).

Given identical data and settings, our chart's output should match Lightweight Charts pixel-for-pixel in most cases (especially for core types and default styles). Minor differences might occur due to anti-aliasing or slight coordinate rounding, but we aim to be perceptually the same or better (e.g., our text might be slightly sharper due to different scaling, but we can accept that if it's correct content).

Automation: We'll integrate these tests into CI: - For performance, a special test page can run the scenarios and log results to console. We consider using the browser's Performance API or a custom harness. Possibly, we will use *Playwright* which can simulate input at high speed (for pan/zoom) and record performance metrics (Playwright has context tracing, etc.). - We set thresholds: for example, *FPS* ≥ 58 in scenario B during continuous pan, *no frame* $> 32ms$ perhaps, memory usage $< X MB$ after scenario D. These thresholds ensure we beat the baseline (we could also run the same scenarios on TradingView's library for comparison numbers, but since it's a different lib in possibly a different environment, direct comparison might be complicated; instead, we use absolute targets gleaned from known capabilities of LC: e.g., LC can easily do 1M points with conflation – we ensure we do at least that).

Golden Examples: We'll include some known complex cases as regression: - **Edge case data:** e.g., all equal values (flat line) – ensure autoscale adds margin and crosshair doesn't NaN out (LC had some issues historically with log scale on equal values, we test that). - **Log scale negative values:** ensure we handle gracefully (likely by not plotting or converting to small positive, as LC would likely ignore negative on log). - **Whitespace data:** e.g., a gap in series – verify that gap is indeed blank on chart (no interpolation). - **Switching options:** call `series.applyOptions` at runtime to change style (like change candlestick colors) and ensure chart updates immediately and correctly (no lingering old color). - **Resizing:** programmatically resize from small to large – ensure content scales and no high-DPI blurriness (we can screenshot before/after resize and compare relative sharpness or label correctness).

The harness will produce a report after running: - Performance metrics with pass/fail against thresholds (e.g., "Max frame time 10ms (PASS, under 16ms)"). - Perhaps comparative charts or logs for dev insight (like memory vs time graph). - Visual diffs for any failed image comparisons (with diff images saved for analysis).

Pass/Fail Criteria ("better than"): We define "better than TradingView LC" as: - Able to handle equal or greater data volume at equal or higher FPS. So if LC struggles at ~5M points, we ensure smooth operation at 5M (with conflation, since LC introduced conflation for exactly that). - Lower **input-to-render latency**. E.g., crosshair feels immediate; if LC had a slight lag on huge data, ours should be imperceptible. - **Bundle size** $\leq 60KB$ gz (LC v5 is ~35kB core but plugins add – our aim is under 60kB even with core + typical plugins). We'll measure our built size and ensure it's under threshold (fail CI if above). - **Visual fidelity:** No regression in visual accuracy – all chart features render correctly. If we intentionally improved something (like how overlapping axis labels might be handled better), that's not a regression but a conscious difference; we document those.

We will also incorporate memory leak tests in the harness: create a chart, add/remove series repeatedly or create/destroy chart 100x, and monitor memory (via heap snapshots). There should be no accumulating growth – everything should be cleaned up on `.remove()` (LC expects you call `chart.remove()` to detach – we do same ¹⁷¹, and we test that it indeed releases references).

Continuous Integration: After any major change, we run these scenario tests. If any performance metric degrades (e.g., FPS drops by $>10\%$ in scenario D), that surfaces as a potential regression to investigate.

Visual diff tests ensure a change in drawing code doesn't inadvertently break an appearance (especially important for candlesticks and others where a subtle off-by-one could color a pixel wrongly).

In summary, this harness not only guarantees that we meet our design goals, but also provides confidence to iterate rapidly without fear of silent performance regressions. It effectively quantifies "better than TradingView LC" in concrete numbers and images.

7. Migration Strategy

For a seamless transition from the existing chart to our new architecture, we propose a phased migration with an **adapter layer** to minimize disruption:

Step 1: Compatibility Adapter: We implement an adapter that wraps our new chart API to mimic the old chart's interface. If the existing chart's API was modeled after TradingView LC (as implied), then our API is already very close. But if there are differences (method names, behaviors), the adapter can translate calls. For example, if existing code calls `oldChart.addCandlestickSeries(options)`, our adapter can call `createChart.addSeries('Candlestick', options)` and return a wrapped series. This adapter allows running existing integration code with minimal changes while actually using the new internals. We should document any small differences (e.g., our crosshair event might provide data in a Map vs an object; the adapter can convert it to old format if needed, or we instruct user to update usage).

Step 2: Incremental Feature Replacement: We recommend migrating feature by feature: - Start using the new **rendering engine** with existing data feed. For instance, plug the existing data stream into our chart (via adapter or directly). This immediately improves performance, hopefully without changing how data is fetched or managed above. - Migrate **interaction handling**: if the old chart had custom event handlers or logic (like custom zoom buttons that call some internal methods), ensure those now call our chart's methods. Our API often overlaps LC's, so likely minimal changes. - Migrate **plugins/markers**: If the old chart allowed markers or custom drawings differently, implement those through our plugin system or core API. We might maintain the same function signatures (through adapter) so existing marker insertion calls still work.

Parallel Running (if needed): During migration, one could run both charts side by side for comparison (using the same data). Because our system is highly optimized, this likely isn't too heavy for short test runs. Visual output can be compared (maybe not pixel-perfect due to rendering differences, but qualitatively). This helps validate that after migration, all features (like specific custom styling or edge-case data) are still correct.

Data & State Transfer: If the current chart holds a lot of state (like already loaded historical data, current zoom level, etc.), we provide methods to import that into our chart: - We can allow creating a new chart with an initial visible range or scroll position to match the old chart's view. - We can dump series data from old chart (if accessible) and simply call `setData` on our series to preload it. Because our data structures are efficient, this one-time cost is fine.

Phased UI Switch: If possible, gradually enable features of new chart for users: - e.g., first switch to new chart for static charts while leaving old one for real-time if uncertain. But since our goal is full parity and

beyond, it might be fine to switch all at once once testing is done. - A feature flag or environment toggle can allow rolling back if an unexpected issue arises in production (keeping risk low).

Incremental Replacement Plan: 1. **Integration Testing in Sandbox:** Use the adapter to plug new chart in a staging environment that mimics production data feeds (ensure no backlog or performance issue). Monitor performance (should see improvements in CPU/memory). 2. **Beta Release with Fallback:** Release the new chart to a subset of users or behind a toggle. If any problems (visual glitches, or some interactions not working as before), they can switch back to old chart seamlessly. Collect any bug reports. 3. **Full Cutover:** Once confident, remove old chart library from bundle and switch entirely to new. The adapter layer itself can also be gradually removed – ideally, update the codebase to call the new API directly for clarity (especially if we want to use new features or improved patterns). This can be done gradually: e.g., refactor modules that were using old chart one by one to use new API, test, and then drop adapter.

Known Differences and Solutions: - If the old chart didn't support multi-pane and our new does, no issue (just unused feature). If the old had multi-pane differently, we map accordingly. - If any option names changed (we tried not to, but maybe minor differences), ensure adapter covers them or alias them. - The visual differences like how crosshair magnet mode works (LC's magnet snapped to close only, our default might allow OHLC if configured) – to maintain exact behavior, set `crosshair.mode` to match old behavior (Magnet or MagnetOHLC). - For any deprecated behaviors (maybe old chart had something not recommended), we either continue support via adapter (if harmless) or document a needed change.

Performance Consideration in Migration: The user's existing chart might have built workarounds due to previous performance limitations (like limiting data or decimating externally). With our new chart, those might be unnecessary or could be removed to fully realize performance gains. After migrating, we can simplify upstream code – e.g., if they had logic to throttle updates to the old chart, maybe they can increase frequency now.

Verifying Migration Success: After switch: - Run all the existing unit tests (if any) for chart behavior – they should all pass with new chart. If differences are intended, update tests accordingly. - Do visual regression tests comparing output of old vs new on key scenarios (we can use our harness to capture old chart image vs new chart image with same data to ensure they match as expected). - Monitor in production environment metrics like CPU usage on user devices, memory usage, and any user feedback (we expect improved smoothness and lower resource usage, which should reflect in things like lower average script timing and fewer frame drop reports in RUM data if collected).

Documentation & Support: Provide internal docs for developers to know how to use new API (though it's similar to old, a quick guide focusing on differences helps). Possibly keep a mapping table of old method -> new method (like our parity matrix can serve as a reference). Since we largely kept API similar, this is straightforward.

Step-by-Step Example: 1. Wrap `createChart` such that it calls our `createChart` under the hood and returns an object that implements all methods the old chart instance had, delegating to new. 2. Swap the library include: drop old library import, include new one + adapter. 3. Test all features (drawing of each series type, adding/removing series, updating data, interactions). Fix any found issues (with adapter or minor option adjustments). 4. Once stable, gradually remove the adapter in favor of using new API directly in code for clarity (the adapter was mainly a crutch for quick migration, long-term better to call our native methods).

Using this strategy, we ensure that the migration is low-risk and that we **retain all functionality during the switch**. Users might notice only improvements (e.g., smoother zoom or clearer text). If anything, we might need to fine-tune styling to match old exactly (like color of grid lines or default font size might differ slightly); those can be set in options to match previous look, so end-users see a seamless visual continuity.

8. Codex Build Roadmap

We outline a milestone-based roadmap to implement this system, facilitating testing at each stage and using our harness to validate as we go. The development is organized in logical increments:

Milestone 1: Project Setup & Core Classes - *Task 1.1: Scaffold Base Classes* – Create empty classes/interfaces for Chart, Pane, Series, TimeScale, PriceScale, DataStore, etc., based on architecture design. Stub out methods (no internal logic yet). *Test:* Ensure the project builds, and a basic `createChart(container)` returns a `ChartApi` object without error. - *Task 1.2: Basic Rendering Loop* – Implement `RenderCoordinator` with `rAF`. Have it simply clear a canvas and fill with a background color each frame (as placeholder). *Test:* Verify `rAF` is running and can be started/stopped with chart lifecycle. - *Task 1.3: TimeScale & PriceScale fundamentals* – Implement linear transform for `TimeScale` (mapping index to x) and `PriceScale` (linear mode only). *Test:* Insert dummy data (e.g., 10 points) in a `DataStore`, manually set `PriceScale` range, and ensure `priceToCoordinate` and `timeToCoordinate` give expected pixel values for a given canvas size.

Milestone 2: Single-Series Chart (Line) - *Task 2.1: DataStore & LineSeries* – Implement `SingleValueData` storage in `DataStore` (sorted by time). Implement `LineSeriesPaneView.draw(ctx)` to draw a simple polyline using current scales. Only support linear scales for now. *Test:* Create a chart with one line series, call `setData` with a small array, and verify a line is drawn (manually or via pixel test for known points). - *Task 2.2: Axes Rendering* – Implement a basic `PriceScale` axis draw (a few ticks: min, max, maybe one mid). Implement `TimeScale` axis draw (tick at first, middle, last for now). *Test:* Check that axes canvases show proper labels matching data range (e.g., if data 10-20, axis shows 10 and 20). - *Task 2.3: Crosshair Basic* – Draw crosshair lines at a given logical position (we can simulate at middle of chart). *Test:* Hardcode crosshair at index 5, verify lines visible and labels show corresponding time/value.

At this point, we have a minimal chart that can display a line series and axes.

Milestone 3: Interaction & Multiple Series - *Task 3.1: InteractionManager* – Implement `mousemove`, `mouseleave` to position crosshair; `mousedown/move` for panning horizontally; `mousewheel` for scroll. *Test:* Manually pan and zoom the line chart – ensure it responds (line moves, axes update). Use performance monitor to ensure no major frame drops with a few thousand points. - *Task 3.2: Multiple Series & Panes* – Allow adding another series (e.g., a second line or histogram). Implement layering: draw series in order added. Implement second Pane: `chart.addPane` creates new canvas and associated `PriceScale`. *Test:* Add two series on same pane (different scales perhaps) and ensure both render. Add a second pane with one series – ensure it stacks below and can be resized via `IPaneApi.setHeight` (set heights and call `chart.resize` to layout). - *Task 3.3: Candlestick & Bar Series* – Implement `CandlestickPaneView` and `BarPaneView` drawing. Reuse logic for OHLC data. *Test:* Plot known OHLC data (including up and down candles) and verify drawn correctly (maybe compare to reference image or manual visual check). - *Task 3.4: Area & Baseline Series* – Implement Area (fill under line) and Baseline (two-tone area). *Test:* Area with sample data – verify fill color gradient. Baseline – set `baseValue` and test data above/below – verify color split at baseline (e.g., baseline = 0, positive area green, negative red). - *Task 3.5: Histogram Series* – Implement vertical bars drawing for

histogram. *Test:* Use e.g. volume data to see bars drawn, test custom per-point color. - **Task 3.6: Autoscale & Margins** – Implement autoscale calculation: PriceScale finds min/max of visible points (each DataStore can provide visible range min/max quickly by binary search). Add margin (top/bottom). *Test:* Zoom/pan to different segments and ensure price axis adjusts properly (with ~10% margin by default). - **Task 3.7: Logarithmic Scale & Modes** – Implement log scale transform, plus Percentage/IndexedTo100 modes. *Test:* Switch a series priceScale to log with a range spanning an order of magnitude – verify spacing (we can calculate expected log positions). Test percentage mode on a known series (first visible value becomes 0%, etc.). - **Task 3.8: Localization & Time Formats** – Implement time zone offset application and `timeVisible` / `secondsVisible` options. Possibly use Intl for formatting. *Test:* For intraday data, toggle `secondsVisible` and confirm axis labels include seconds or not accordingly ¹¹⁴.

By end of Milestone 3, we support all series types and core interactions.

Milestone 4: Plugins & Advanced Features - **Task 4.1: Series Markers Plugin** – Implement series markers (shapes on series at specific indices). Possibly follow LC's plugin separation: internal class that draws on top of series. Provide `SeriesApi.setMarkers` to use it. *Test:* Add markers (e.g., `arrowUp` at specific bar) and verify they show above that bar. - **Task 4.2: UpDownMarkers Plugin** – Implement specialized up/down markers (if required, could be similar to series markers but perhaps automatically marking ups/downs). - **Task 4.3: Watermark (Text/Image)** – Implement a `PanePrimitive` for watermark. `TextWatermark`: draw centered text with given font and opacity ¹⁴⁸. `ImageWatermark`: draw an image similarly. Provide `ChartApi` methods `addWatermark`. *Test:* Add text watermark, verify it's behind series (draw first). - **Task 4.4: Custom Series Hook** – Define `ICustomSeriesRenderer` interface and make `chart.addCustomSeries` register it. *Test:* Create a trivial custom series (e.g., draws a triangle at each data point) implement and add – verify it draws. This ensures our architecture is truly extensible.

Milestone 5: Performance Tuning - **Task 5.1: Conflation Implementation** – Implement data conflation in `DataStore`: when requested, aggregate data in blocks such that each output point ~ covers <0.5px interval ¹⁷². Provide `ChartOption` `enableConflation` and series override `conflationThresholdFactor`. *Test:* Create a large dataset (e.g., sine wave 100k points). Compare rendering with conflation on vs off at far zoom – off should be very slow or dense (maybe skip drawing due to our detection), on should be smooth and output fewer points. Also ensure shape looks correct (smoothed). - **Task 5.2: Kinetic Scrolling** – Implement inertial scroll on touch. *Test:* Simulate a quick swipe on chart with devtools sensors – ensure chart continues moving after finger “releases” and then stops at correct position. - **Task 5.3: OffscreenCanvas Support (optional)** – Set up a worker and `OffscreenCanvas` pipeline. *Test:* Using a supporting browser, verify that heavy drawing (lots of points) doesn't block main thread (we can detect by running a main thread script concurrently). - **Task 5.4: Benchmark Automation Integration** – Write scripts (possibly in Playwright) to run the scenarios. *Test:* Ensure that running `yarn benchmark` or similar yields performance numbers and possibly compare to threshold, failing if above target times.

Milestone 6: Testing & Stabilization - **Task 6.1: Comprehensive Unit Tests:** Write unit tests for each module: - `DataStore`: insertion, removal, binary search. - `TimeScale`: converting times (esp. test daylight savings boundaries if timezone shifts). - `PriceScale`: test each mode mapping a known set of values to normalized `[0, height]`. - `SeriesPaneView` classes: we can't easily verify drawing by unit test without canvas, but we can test their coordinate calculations. Alternatively, use a headless canvas to draw and sample pixels at known positions. - Interaction logic: simulate events on `InteractionManager` with a stubbed chart and check internal state changes (like crosshair position, scroll position, etc.). - **Task 6.2: Visual Regression Tests:** Use Puppeteer to generate chart images for predefined data sets (covering each series type, combos,

log scale, etc.) and compare to reference images (either from LC or from a manually verified run of our code). Fix any rendering issues uncovered (alignment, color mismatches, etc.). - **Task 6.3: Documentation & Examples:** Create a set of example charts (similar to LC's examples) to verify everything in a realistic scenario (e.g., candlestick chart with volume on second pane, etc.). These also serve as demo pages. As we build them, we may catch any API rough edges or minor bugs. - **Task 6.4: Performance Regression Tests:** Integrate the performance harness in CI to run on a headless browser. Set conservative thresholds for now (ensuring no severe regression from commit to commit). This ensures we catch inadvertent slowdowns (like if a change introduced an $O(n^2)$ somewhere). - **Task 6.5: Migration Adapter:** If migrating an existing system, create the compatibility layer now and run existing tests (if any) or sample usage through it. Ensure all old API use-cases function.

Milestone 7: Release Prep - *Task 7.1:* Clean up any TODOs, ensure code is commented and documented (for API reference generation). - *Task 7.2:* Final bundle size check. Apply any easy optimizations (e.g., dropping debug code, ensuring Treeshakable structure). - *Task 7.3:* Beta release to a staging environment or selected users for feedback. - *Task 7.4:* Collect feedback/bug reports, fix them, update tests accordingly. Likely minor tweaks at this stage (like adjusting default label font or so to match exactly what we want). - *Task 7.5:* Final performance run of scenarios to verify all targets are met or exceeded: e.g., confirm our chart can handle scenario D at least as smoothly as LC (maybe measure on a typical machine or profile in devtools that CPU usage is lower).

Throughout development, each milestone's deliverables can be validated. By Milestone 3 we already have something comparable to LC's core. Milestones 4 and 5 add the advanced capabilities and performance polish that set us apart.

Roadmap Summary: This plan allows incremental building – at end of each milestone, we have a working chart (with progressively more features) which we can test and demo: - End of M2: basic line chart with axes (foundation). - End of M3: full multi-series chart like LC, ready for heavy testing. - End of M5: optimized and feature-complete beyond LC (with conflation, etc.). - End of M6: thoroughly tested and ready for production integration.

Each step is backed by specific tests (unit or scenario), aligning with our quality approach. By following this roadmap, we steadily move from a barebones chart to a **superior, production-ready system**, all while verifying parity and improvements at each stage.

1 2 153 155 160 161 162 163 172 Releases · tradingview/lightweight-charts · GitHub

<https://github.com/tradingview/lightweight-charts/releases>

3 15 16 145 148 159 164 165 167 Release Notes | Lightweight Charts

<https://tradingview.github.io/lightweight-charts/docs/release-notes>

4 5 7 8 9 10 11 12 13 14 Chart types | Lightweight Charts

<https://tradingview.github.io/lightweight-charts/docs/chart-types>

6 84 86 88 91 93 94 95 96 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 157 169 Interface: PriceScaleOptions | Lightweight Charts

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/PriceScaleOptions>

17 18 20 22 23 24 25 26 27 28 29 30 31 158 **Interface: IPaneApi\ | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/IPaneApi>

19 21 **Panes | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/panes>

32 33 34 35 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 118 166

Series | Lightweight Charts

<https://tradingview.github.io/lightweight-charts/docs/series-types>

36 37 38 39 40 41 43 44 45 46 47 48 151 **Interface: LineStyleOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/LineStyleOptions>

42 129 130 133 **Interface: CrosshairLineOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/CrosshairLineOptions>

72 73 74 75 76 77 78 79 80 81 82 85 **Interface: SeriesOptionsCommon | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/SeriesOptionsCommon>

83 87 **Type alias: OverlayPriceScaleOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/type-aliases/OverlayPriceScaleOptions>

89 90 92 97 **Enumeration: PriceScaleMode | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/enumerations/PriceScaleMode>

113 **Interface: TimeScaleOptions | Lightweight Charts - GitHub Pages**

<https://tradingview.github.io/lightweight-charts/docs/3.8/api/interfaces/TimeScaleOptions>

114 **Interface: TimeScaleOptions | Lightweight Charts - GitHub Pages**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/TimeScaleOptions>

115 **How to set a custom time zone for Lightweight Charts?**

<https://stackoverflow.com/questions/65785045/how-to-set-a-custom-time-zone-for-lightweight-charts>

116 117 119 120 121 123 149 156 **Time scale | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/time-scale>

122 **Type alias: Time | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/type-aliases/Time>

124 128 131 **Interface: CrosshairOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/CrosshairOptions>

125 126 127 132 134 150 **Enumeration: CrosshairMode | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/enumerations/CrosshairMode>

135 136 138 139 **Interface: HandleScrollOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/HandleScrollOptions>

137 **UNPKG**

<https://app.unpkg.com/lightweight-charts@5.0.2/files/dist/typings.d.ts>

140 **<https://cdn.jsdelivr.net/npm/lightweight-charts@5....>**

<https://cdn.jsdelivr.net/npm/lightweight-charts@5.0.8/dist/typings.d.ts>

141 142 **Interface: AxisPressedMouseMoveOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/AxisPressedMouseMoveOptions>

143 144 146 147 **Interface: KineticScrollOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/KineticScrollOptions>

152 **Interface: PriceChartOptions | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/PriceChartOptions>

154 **Event when data is updated in the series · Issue #530 - GitHub**

<https://github.com/tradingview/lightweight-charts/issues/530>

168 **Function: createSeriesMarkers() | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/functions/createSeriesMarkers>

170 171 **Interface: IChartApi | Lightweight Charts**

<https://tradingview.github.io/lightweight-charts/docs/api/interfaces/IChartApi>